Facultad de Ingeniería en Sistemas de la Información y Ciencias de la Computación

-VI Ciclo

-Sección: A

-Base de Datos I

Inge. Yoel Fransua Monzón Estrada

17 de octubre de 2026



INFORME DE PRÁCTICA TÉCNICA Y EVIDENCIAS
TAREA NO. 7: LÓGICA DE NEGOCIO Y TRAZABILIDAD EN POSTGRESQL

Estudiante:	Robert Gameros
Repositorio Oficial GitHub:	**https://github.com/RobertGameros/S15-Procedimientos-almacenados-funciones-escalares-y-triggers.git**
Motor de Base de Datos:	PostgreSQL 18.6 (Desplegado con Docker Compose en postgres:18-alpine)
Entorno y Cliente SQL:	JetBrains DataGrip (PostgreSQL JDBC Driver ver. 42.7.13)
Base de Datos Base:	Chinook Database (Luis Rocha) — Catálogo de Música Digital (track)

Introducción y Contexto del Caso de Negocio

Una tienda digital internacional de música requiere actualizar los precios de venta de su catálogo de pistas de audio de forma controlada, segura y con estricta trazabilidad forense. Para dar cumplimiento a las mejores prácticas de ingeniería de software en bases de datos relacionales, el sistema debe resolver tres necesidades operativas críticas:

• 1. Cálculo de Proyección sin Efectos Colaterales: Antes de confirmar o persistir cualquier ajuste porcentual sobre el catálogo, el área de finanzas debe ser capaz de evaluar el precio resultante de manera matemática pura, sin riesgo de alterar registros en disco ni consultar estados mutables.

• 2. Orquestación y Validación de Lógica de Negocio: Al autorizar un ajuste, el sistema debe verificar exhaustivamente la validez de los parámetros de entrada y comprobar la existencia real del producto o pista musical en el catálogo antes de emitir la instrucción de actualización.

• 3. Auditoría Automática e Ineludible: Cada cambio efectivo de precio debe quedar asentado de forma autónoma e instantánea en una bitácora histórica, almacenando el valor anterior, el nuevo valor, la estampa de tiempo exacta y el usuario de base de datos responsable de la operación, sin depender de que el invocador recuerde insertar el registro de auditoría.

Regla de Integridad Financiera

Principio de Inmutabilidad Histórica: En estricto apego a las reglas de integridad financiera, las facturas previamente emitidas en el catálogo Chinook (tabla invoice_line) no deben modificarse en ninguna circunstancia. El ajuste porcentual afecta única y exclusivamente al precio actual de catálogo de la tabla track.

Arquitectura Desacoplada y Tabla Síntesis de Objetos

Para garantizar alta cohesión y bajo acoplamiento, la solución se diseñó en tres capas lógicas independientes dentro del motor de base de datos PostgreSQL 18. Cada objeto resuelve una responsabilidad única bien delimitada:

Objeto	Tipo de Objeto	Parámetros	Retorno / Efecto	Responsabilidad Principal
calcular_precio_ajustado	Función Escalar (PL/pgSQL IMMUTABLE)	precio_actual numeric, porcentaje numeric	numeric (valor atómico a 2 decimales)	Cómputo puramente matemático: valida rangos [-50, 100], precio > 0, redondea a 2 decimales y rechaza resultado <= 0. Cero I/O sobre tablas.
aplicar_ajuste_precio	Procedimiento Almacenado (CREATE PROCEDURE)	p_id_producto integer, p_porcentaje numeric	VOID (UPDATE unit_price en track)	Orquestación de negocio: valida identificador, verifica existencia con IF NOT FOUND, reutiliza obligatoriamente la función escalar y ejecuta UPDATE.
fn_auditar_cambio_precio	Función Trigger (PL/pgSQL)	Ninguno (Variables especiales OLD y NEW)	TRIGGER	Función de soporte que evalúa si OLD.unit_price IS DISTINCT FROM NEW.unit_price e inserta la tupla en auditoria_precios.
trg_auditar_cambio_precio	Disparador a Nivel de Fila (AFTER UPDATE)	Evento: AFTER UPDATE OF unit_price ON track	Dispara fn_auditar_cambio_precio()	Garantiza la trazabilidad autónoma ante modificaciones de precio, sea mediante CALL o sentencias UPDATE directas en SQL.
auditoria_precios	Tabla Relacional de Auditoría	N/A	Almacén histórico inmutable	Persiste id_auditoria (PK), id_producto (FK conceptual), precio_anterior, precio_nuevo, fecha_cambio y usuario_bd.

Consulta 1: Preparación del Catálogo y Esquema de Auditoría

La primera fase técnica se implementó en el archivo 01_esquema_y_datos.sql (disponible también como sql/01_preparacion.sql). Comprende la verificación de la versión del motor PostgreSQL 18.6 desplegado en Docker, el aseguramiento del catálogo Chinook, la creación de la tabla de trazabilidad auditoria_precios y la vista de compatibilidad productos.

3.1. Verificación de Versión del Motor PostgreSQL 18.6

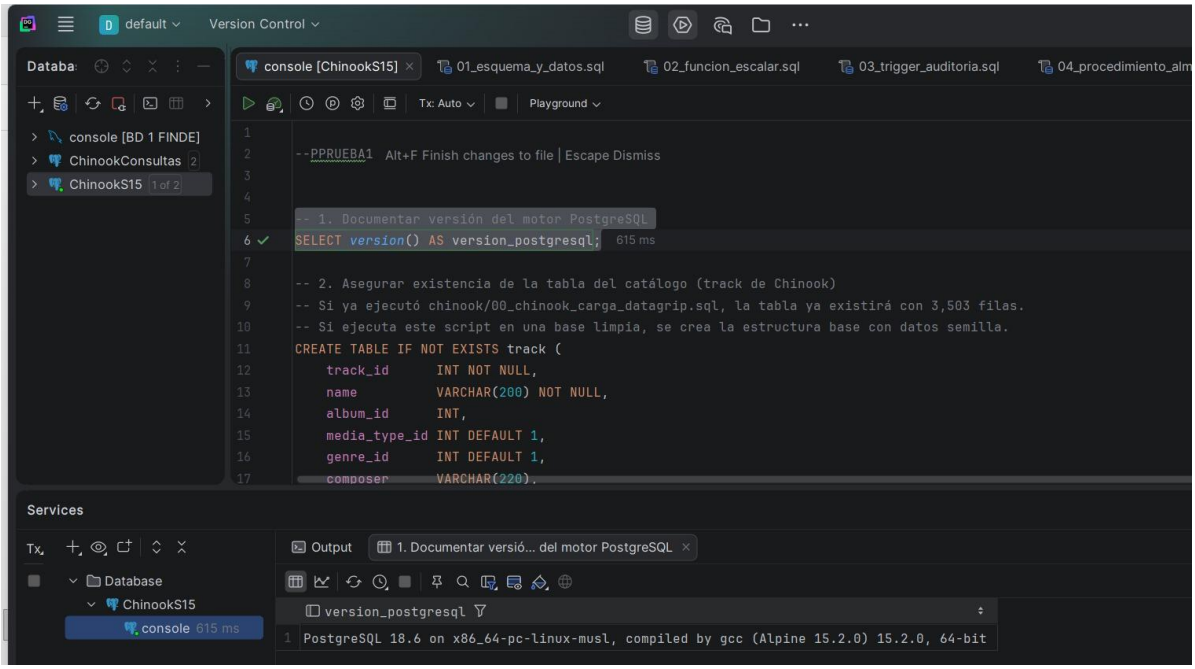


Figura 1: Verificación de la versión del motor de base de datos en DataGrip: PostgreSQL 18.6 on x86_64-pc-linux-musl.

Análisis Técnico: La consulta SELECT version(); ejecutada sobre la conexión 'Chinooks15' ratifica la operación sobre PostgreSQL 18.6 ejecutándose dentro de un contenedor Docker aislado sobre Alpine Linux de 64 bits. Esta evidencia técnica satisface el primer requerimiento de reproducibilidad del entorno de evaluación.

3.2. Catálogo Chinook (track) y Datos Semilla

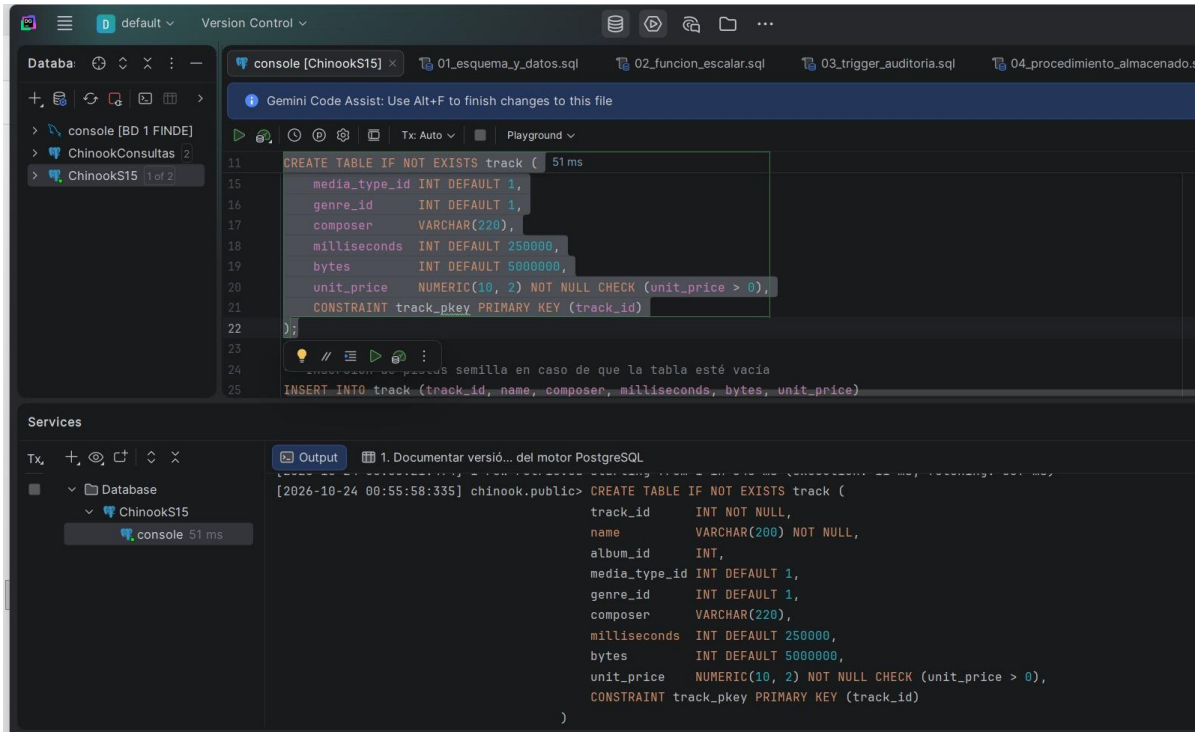


Figura 2: Definición DDL y creación defensiva de la tabla del catálogo track con restricción CHECK (unit_price > 0).

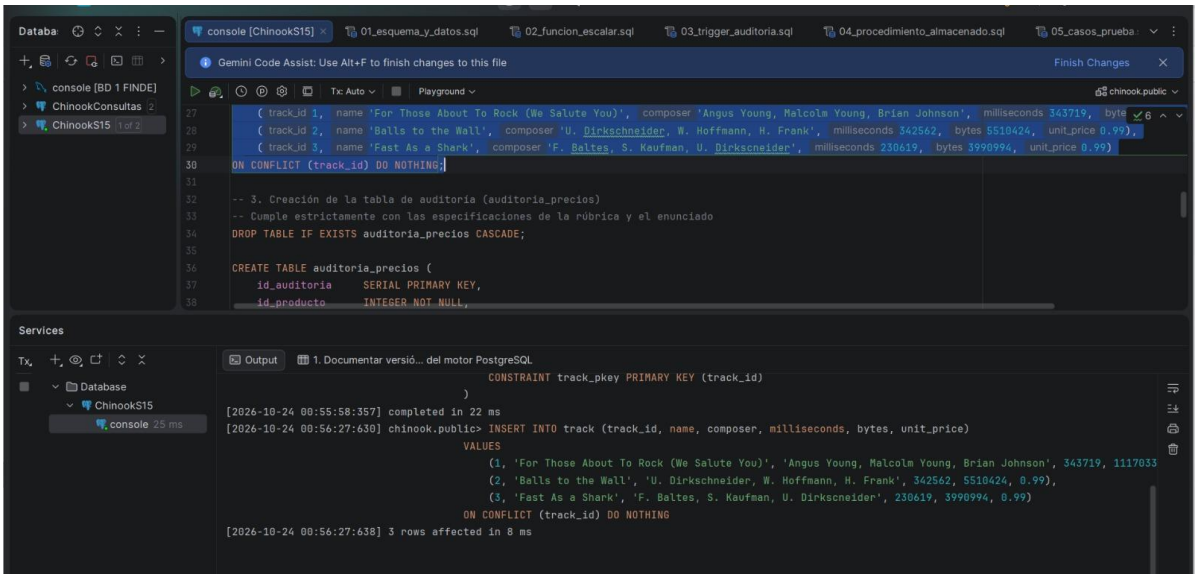


Figura 3: Inserción de las pistas musicales semilla (track_id = 1, 2 y 3) con cláusula ON CONFLICT DO NOTHING (3 rows affected).

Análisis Técnico: Se verifica la disponibilidad de la tabla track del catálogo Chinook con columnas clave track_id (PK), name y unit_price NUMERIC(10,2). En caso de desplegarse en una base limpia, el script inserta de forma autónoma tres pistas semilla con precio unitario 0.99, asegurando que la práctica sea 100% reproducible.

3.3. Creación de la Tabla de Auditoría (auditoria_precios)

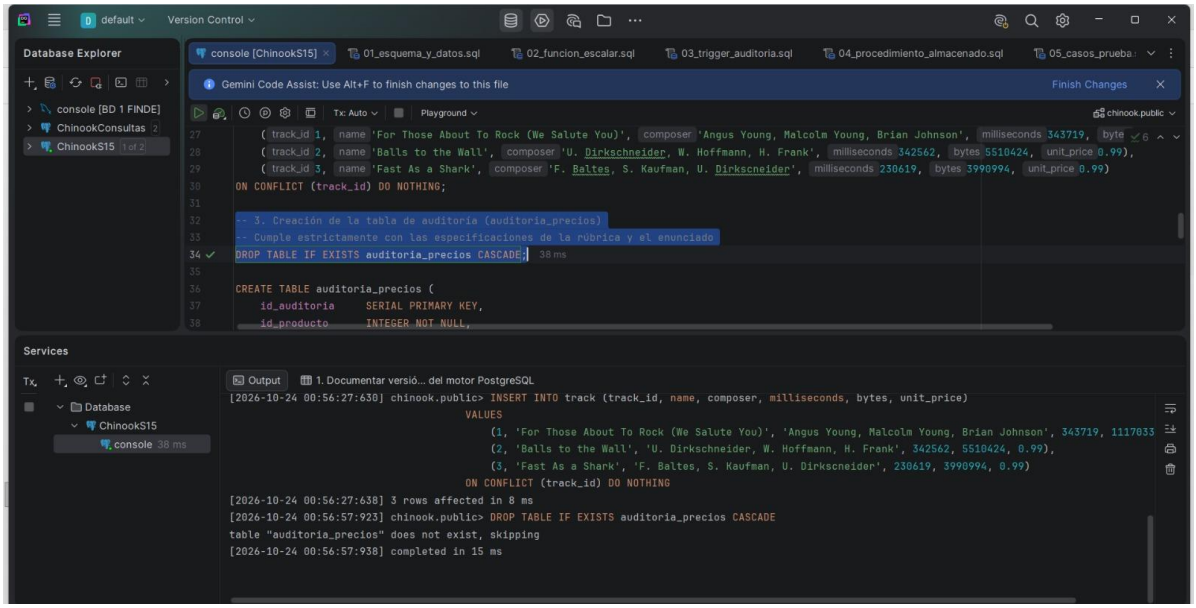


Figura 4: Eliminación segura previa (DROP TABLE IF EXISTS auditoria_precios CASCADE) para garantizar idempotencia.

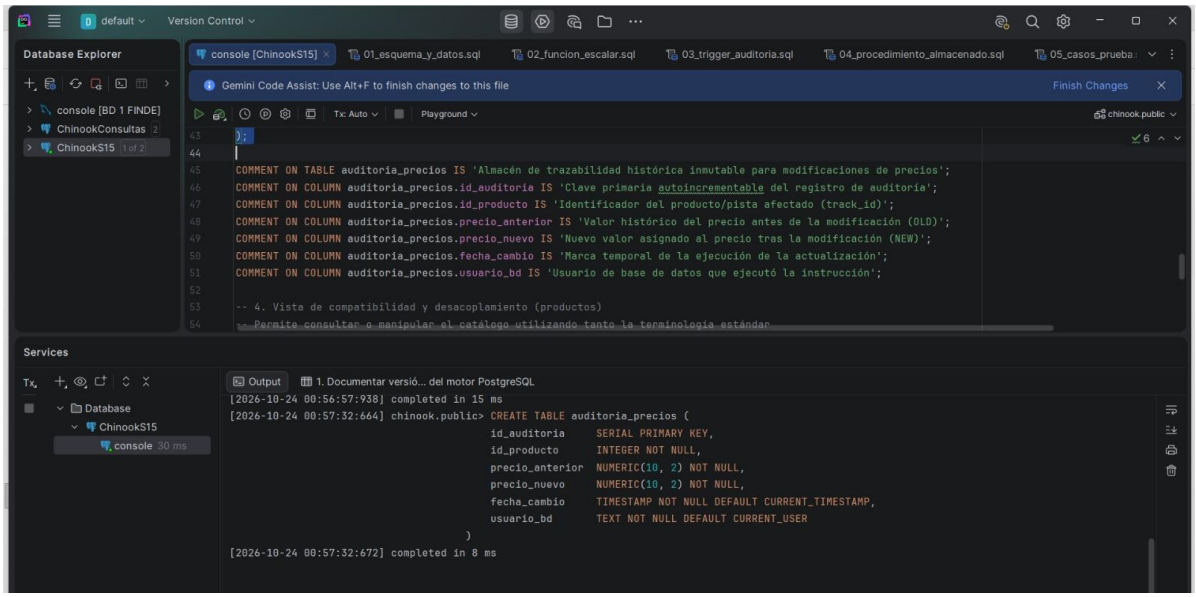


Figura 5: Creación de la tabla auditoria_precios con tipos de datos NUMERIC(10,2), estampa temporal y usuario por defecto.

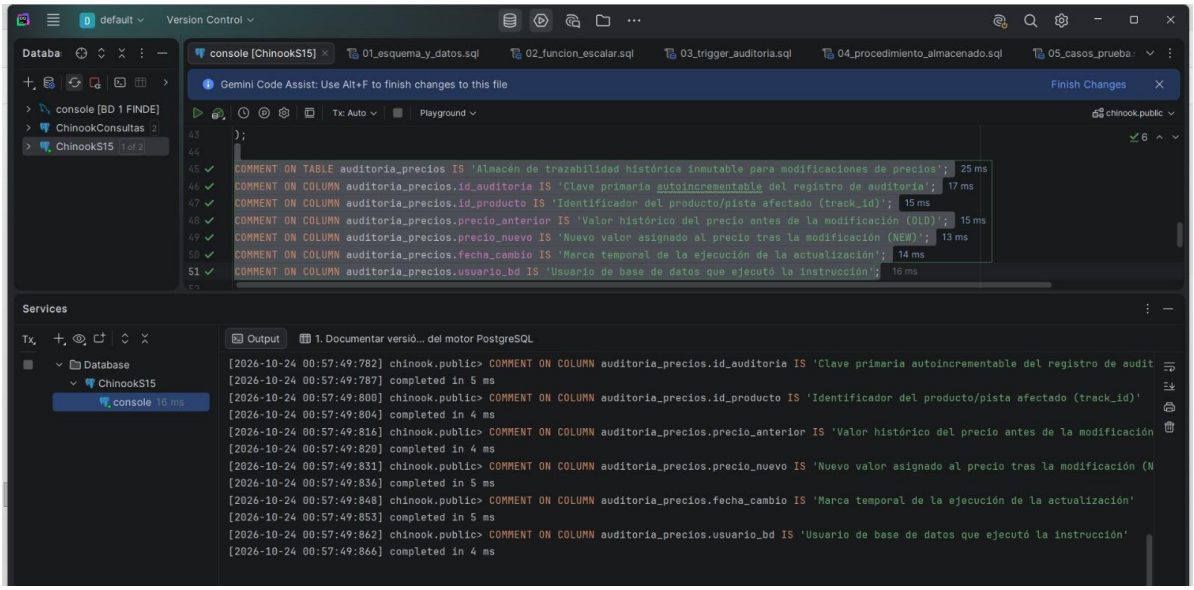


Figura 6: Documentación técnica en el catálogo de PostgreSQL mediante sentencias COMMENT ON TABLE y COMMENT ON COLUMN.

Análisis Técnico: La tabla auditoria_precios se estructuró con los campos obligatorios: id_auditoria (SERIAL PRIMARY KEY), id_producto (INTEGER NOT NULL), precio_anterior (NUMERIC(10,2) NOT NULL), precio_nuevo (NUMERIC(10,2) NOT NULL), fecha_cambio (TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP) y usuario_bd (TEXT NOT NULL DEFAULT CURRENT_USER). Se añadieron metadatos en el catálogo de PostgreSQL para documentar cada columna.

3.4. Vista de Compatibilidad y Estado Inicial Limpio

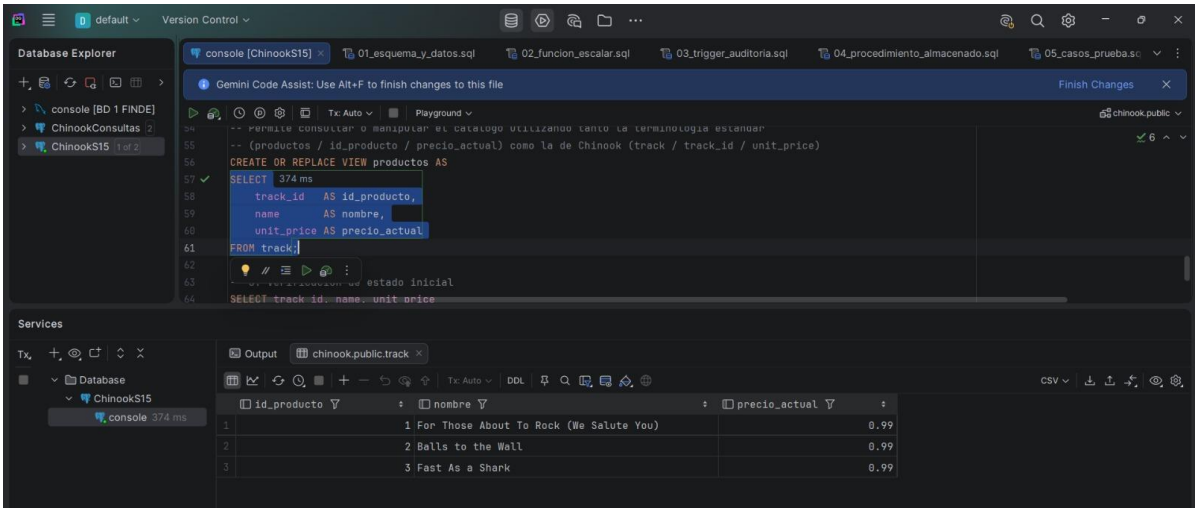


Figura 7: Creación de la vista actualizable productos para desacoplar nombres estándar de negocio del esquema físico de Chinook.

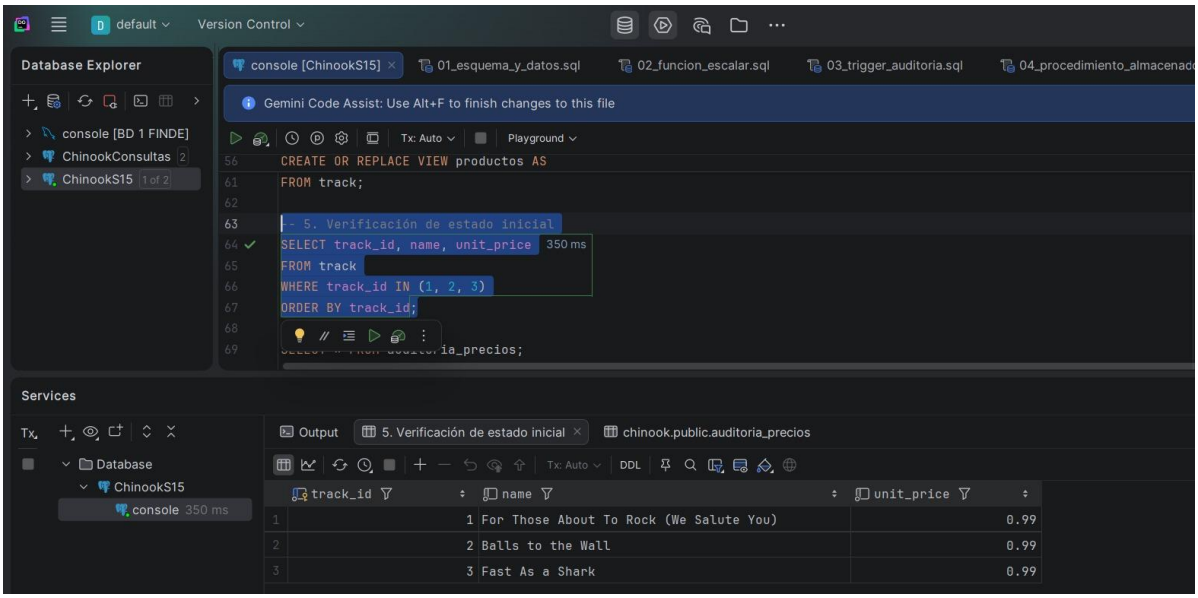


Figura 8: Inspección de los productos testigo del catálogo antes de iniciar cualquier manipulación (Pistas 1, 2 y 3 con precio 0.99).

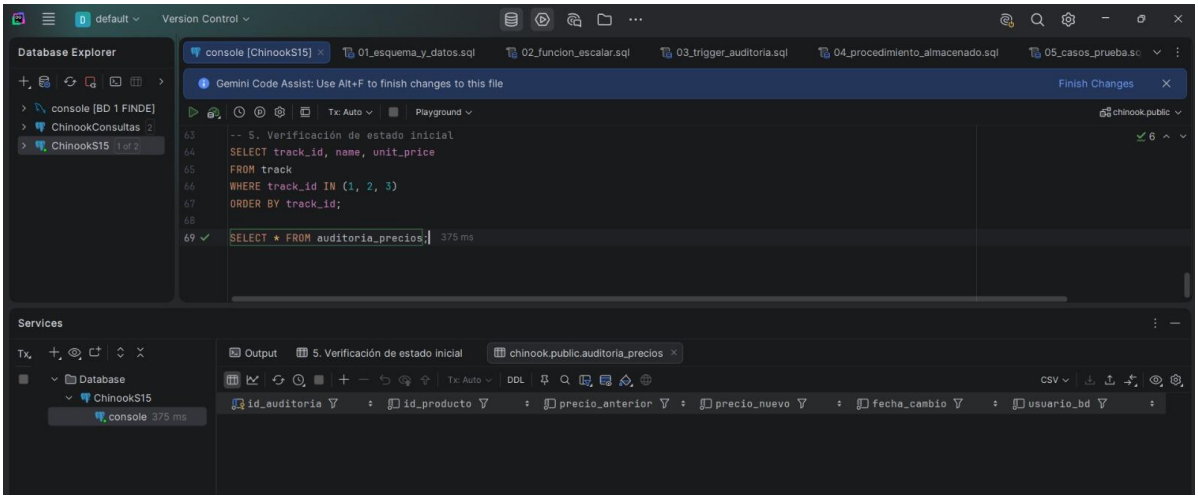


Figura 9: Comprobación del estado inicial de auditoria_precios: tabla completamente vacía (0 registros), estableciendo la línea base.

Análisis Técnico: La vista productos mapea 1 a 1 las columnas de track (track_id -> id_producto, name -> nombre, unit_price -> precio_actual), permitiendo operar bajo el estándar de 'productos' sin alterar el motor. Por su parte, la Figura 9 demuestra que auditoria_precios inicia vacía, lo cual es estrictamente necesario para certificar que cualquier registro posterior provendrá exclusivamente de los triggers de actualización.

Consulta 2: Función Escalar calcular_precio_ajustado

La función matemática se implementó en el archivo 02_funcion_escalar.sql. Representa la primera capa del sistema, orientada a cómputo matemático puro sin efectos colaterales.

Fundamentación Teórica de Diseño

Justificación Teórica de Función Escalar vs. Tabular:

Esta rutina se define como función ESCALAR (retorna un único valor escalar atómico de tipo NUMERIC) y NO como una función tabular (SETOF o TABLE). Esto se debe a que su responsabilidad exclusiva es computar el ajuste de un solo valor monetario a partir de un porcentaje. Al ser escalar, puede componerse libremente como una expresión algebraica dentro de sentencias SELECT, cláusulas WHERE, asignaciones de variables en PL/pgSQL y sentencias UPDATE. Se declaró con volatilidad IMMUTABLE al ser puramente determinista y no acceder a tablas ni estados del sistema.

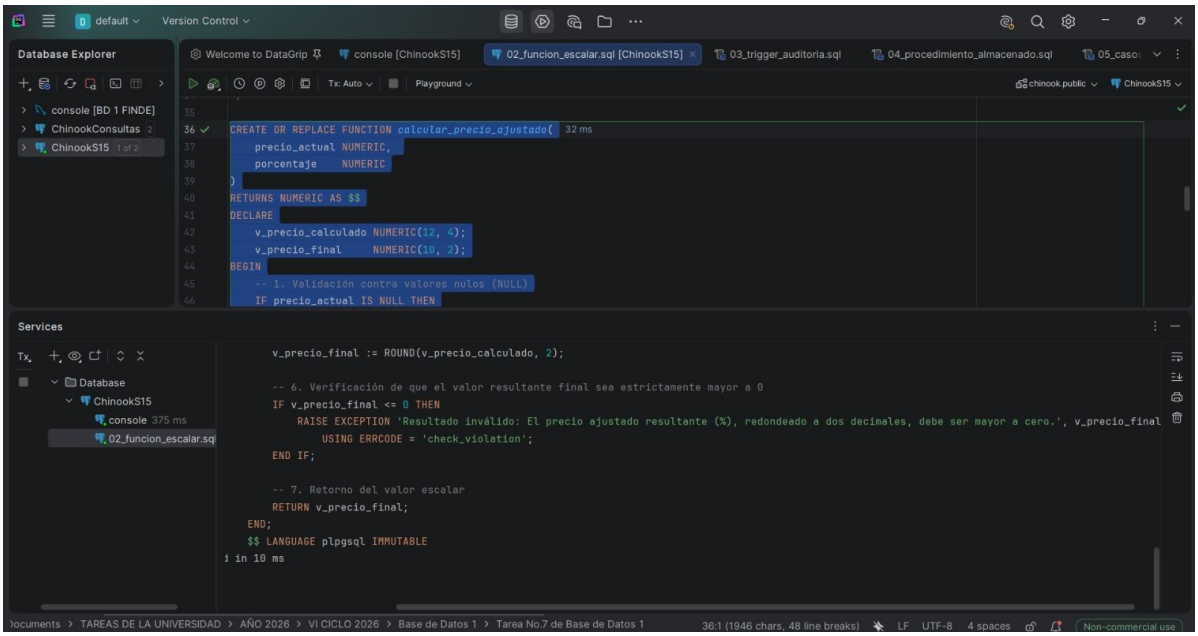


Figura 10: Compilación exitosa de la función escalar calcular_precio_ajustado en DataGrip (completed in 32 ms).

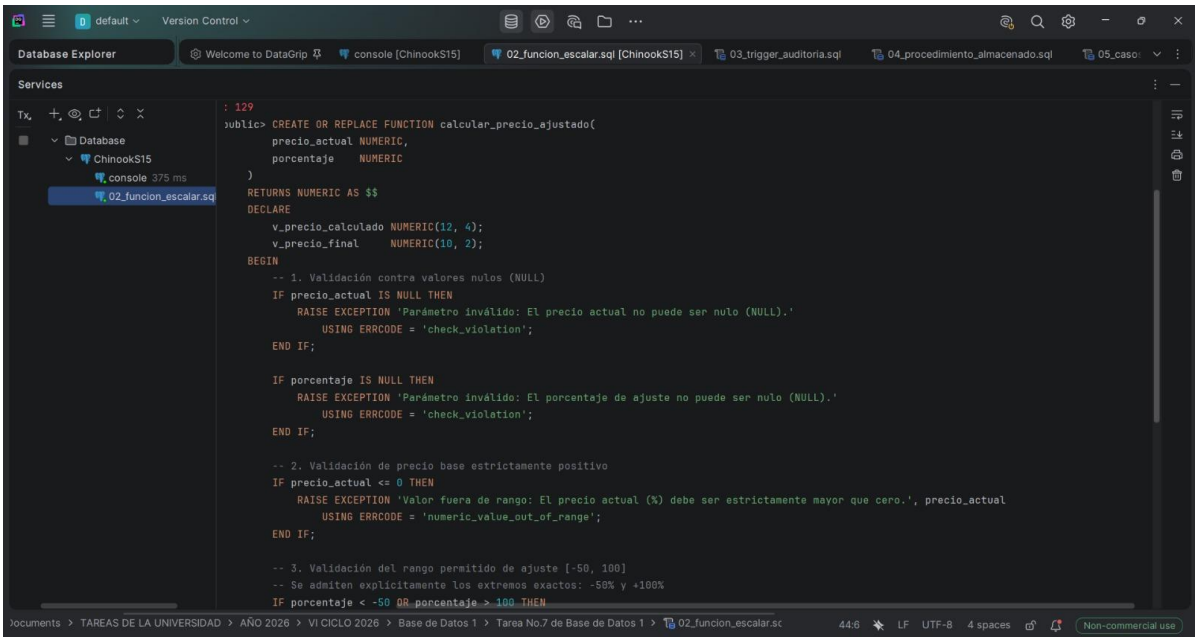


Figura 11: Bloque de validaciones defensivas: rechazo de NULL, validación de precio_actual > 0 y porcentaje en [-50, 100].

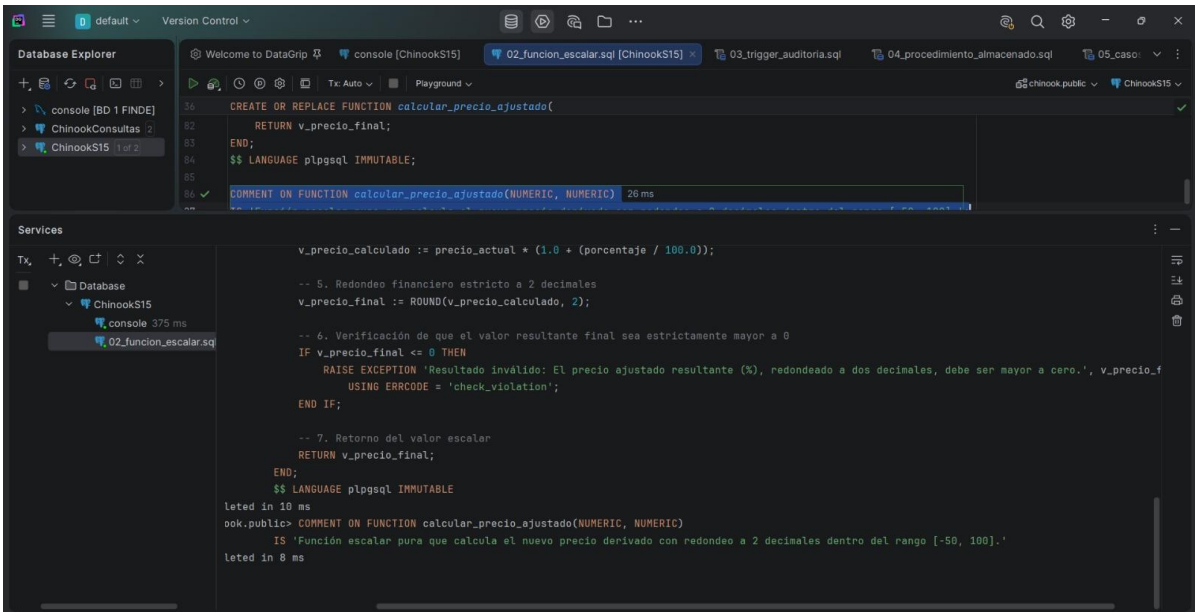


Figura 12: Bloque de cálculo con ROUND a 2 decimales, validación de resultado > 0, declaración IMMUTABLE y comentario del objeto.

Reglas de Negocio Implementadas: La función calcular_precio_ajustado aplica la fórmula matemática precio_actual * (1 + (porcentaje / 100.0)) y redondea estrictamente a dos decimales con ROUND. Si alguno de los argumentos es NULL o si el precio base es menor o igual a cero, emite excepciones con códigos check_violation o numeric_value_out_of_range. Igualmente, restringe los porcentajes de ajuste al rango [-50, 100], permitiendo aumentos de hasta el 100% y descuentos de hasta el 50%.

Consulta 3: Disparador y Función Trigger de Auditoría

La segunda capa arquitectónica se desarrolló en el archivo 03_trigger_auditoria.sql (y sql/04_trigger_y_funcion.sql). Se compone de una función trigger y un disparador a nivel de fila asociado al evento AFTER UPDATE sobre la tabla del catálogo.

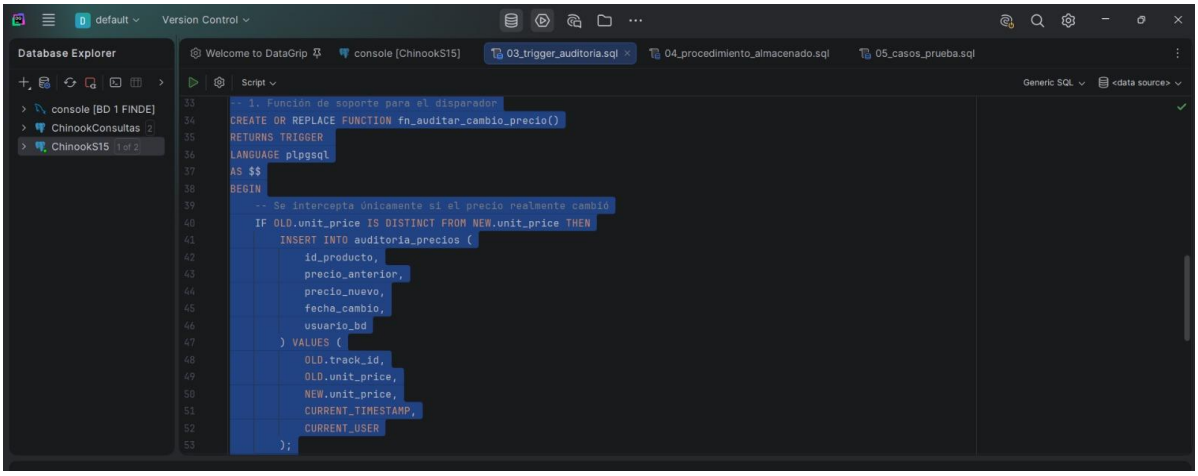


Figura 13: Definición de la función de soporte fn_auditar_cambio_precio() con evaluación condicional IS DISTINCT FROM.

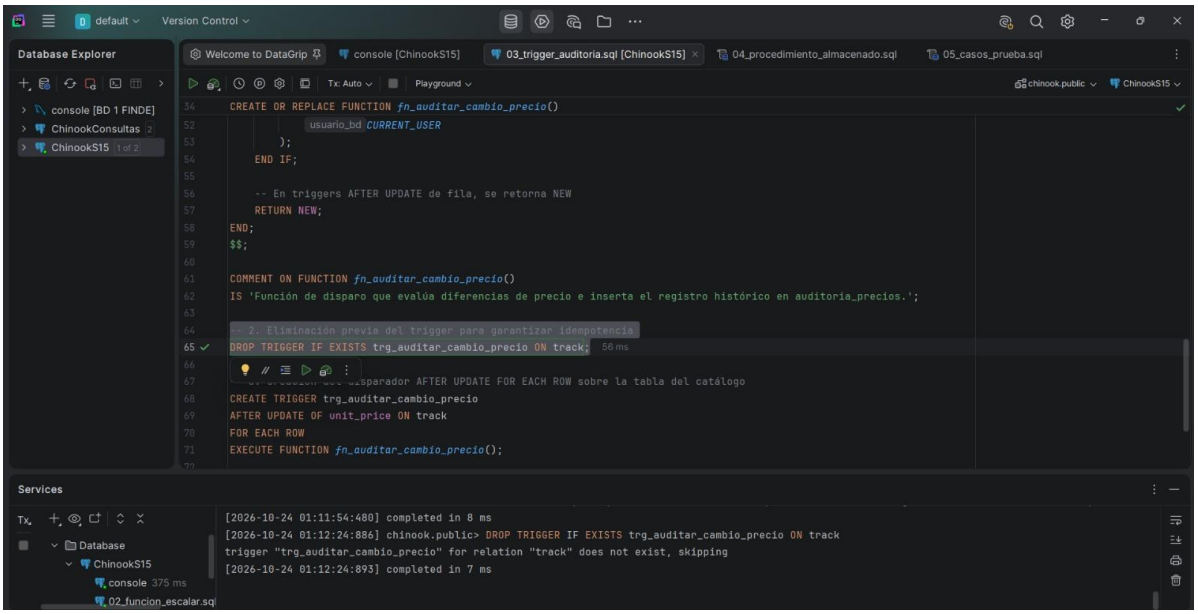


Figura 14: Eliminación segura del trigger preexistente con DROP TRIGGER IF EXISTS para mantener idempotencia de despliegue.

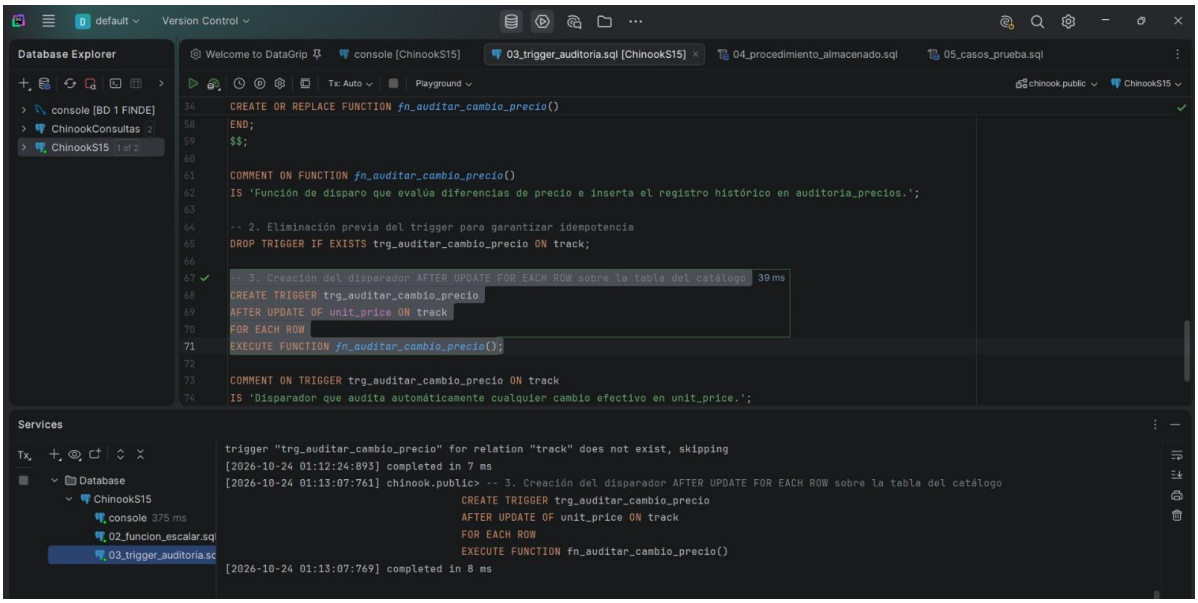


Figura 15: Creación del disparador trg_auditar_cambio_precio sobre la columna unit_price de la tabla track (completed in 8 ms).

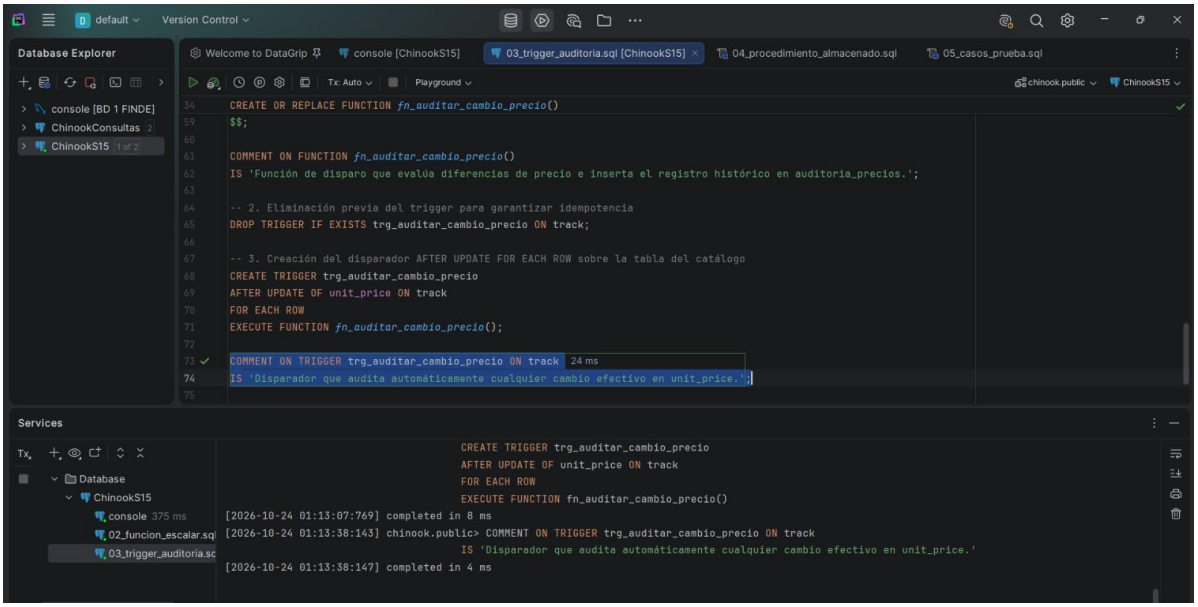


Figura 16: Documentación técnica del trigger asociada en el catálogo de PostgreSQL mediante COMMENT ON TRIGGER.

Mecanismos de Protección Implementados:

1. Discriminación con IS DISTINCT FROM: La condición IF OLD.unit_price IS DISTINCT FROM NEW.unit_price THEN previene que sentencias UPDATE que asignen el mismo precio preexistente generen registros espurios o redundantes en auditoria_precios.

2. Trazabilidad Automática e Ineludible: El disparador opera a nivel del motor relacional. Cualquier sentencia UPDATE (provenga de una aplicación externa, de un procedimiento o de un script interactivo en consola) dispara el registro histórico con marca temporal de servidor

(CURRENT_TIMESTAMP) y usuario de sesión (CURRENT_USER).
3. Orden de Compilación: Este script se ejecutó antes del procedimiento almacenado para que las pruebas posteriores del procedimiento activaran la auditoría de forma natural.

Consulta 4: Procedimiento Almacenado aplicar_ajuste_precio

El procedimiento orquestador se implementó en el archivo 04_procedimiento_almacenado.sql (y sql/03_procedimiento.sql). Se define mediante la sintaxis moderna CREATE OR REPLACE PROCEDURE y se ejecuta mediante la sentencia CALL.

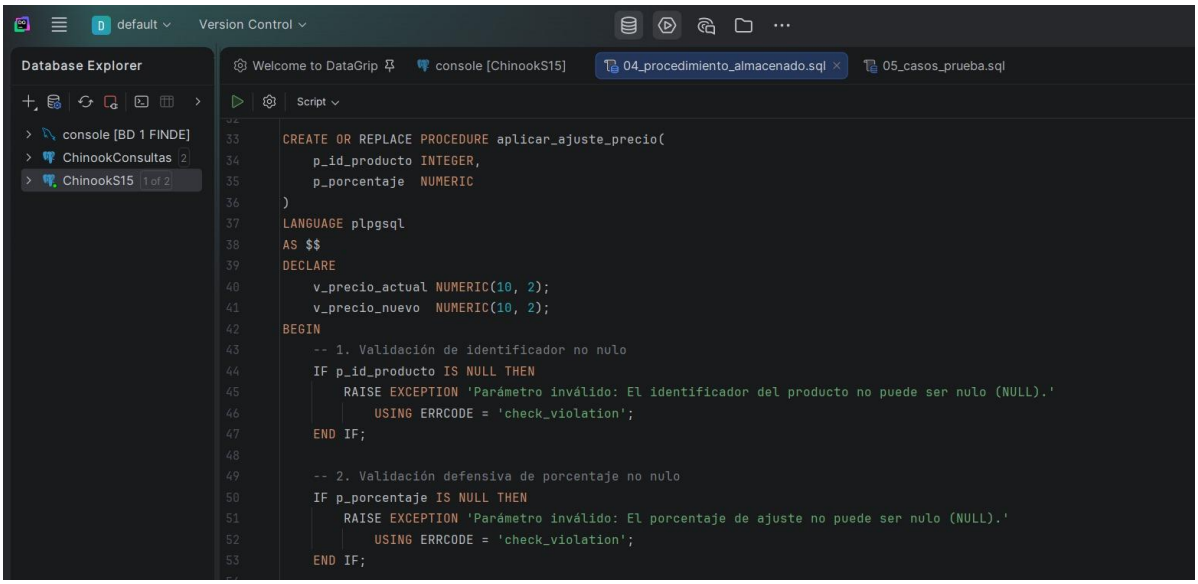


Figura 17: Definición del procedimiento aplicar_ajuste_precio y validación defensiva de identificador y porcentaje no nulos.

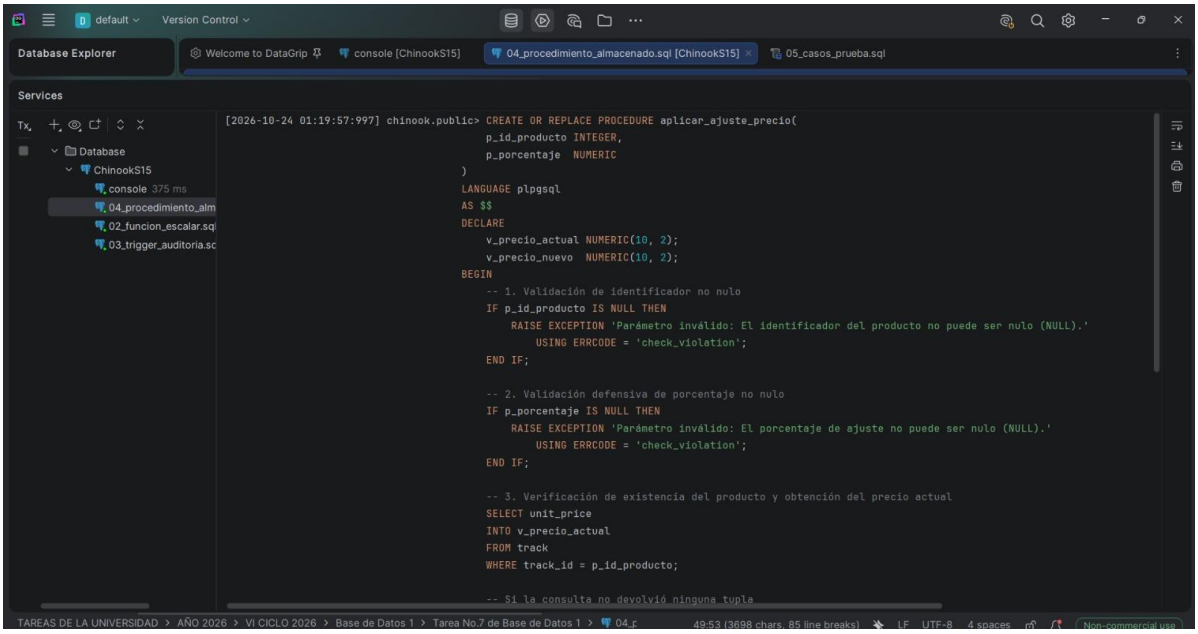


Figura 18: Verificación de existencia del producto en el catálogo mediante SELECT INTO y evaluación de excepción IF NOT FOUND.

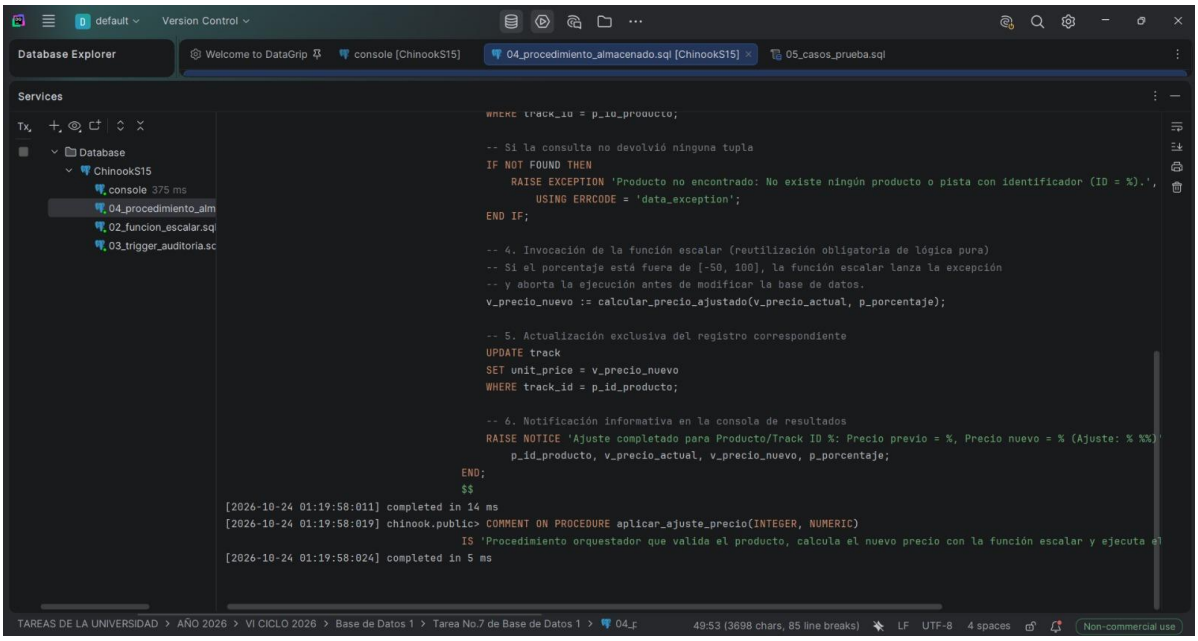


Figura 19: Reutilización de calcular_precio_ajustado, actualización con WHERE track_id y compilación exitosa (completed in 14 ms).

Aspectos Arquitectónicos Clave:

• Reutilización de Código sin Duplicidad: El procedimiento NO repite la fórmula matemática del porcentaje ni sus comprobaciones de límites. En su lugar, invoca v_precio_nuevo := calcular_precio_ajustado(v_precio_actual, p_porcentaje); delegando la responsabilidad matemática a la función escalar.

• Desacoplamiento de la Bitácora: El procedimiento NO contiene instrucciones INSERT sobre auditoria_precios. Esta separación de responsabilidades asegura que si el procedimiento aborta por error, no quedan registros huérfanos.

• Delegación Transaccional: No fuerza sentencias COMMIT ni ROLLBACK internas, otorgando el control de la transacción a la sesión cliente.

Consulta 5: Batería Completa de los 8 Casos de Prueba Evaluables

Las pruebas de aceptación y verificación integral se diseñaron en el script 05_casos_prueba.sql (y sql/05_pruebas.sql). Cada caso fue ejecutado de forma controlada en JetBrains DataGrip, demostrando tanto los resultados numéricos exitosos como las excepciones controladas del motor.

7.0. Fase 0: Inicialización y Establecimiento de Precios Base

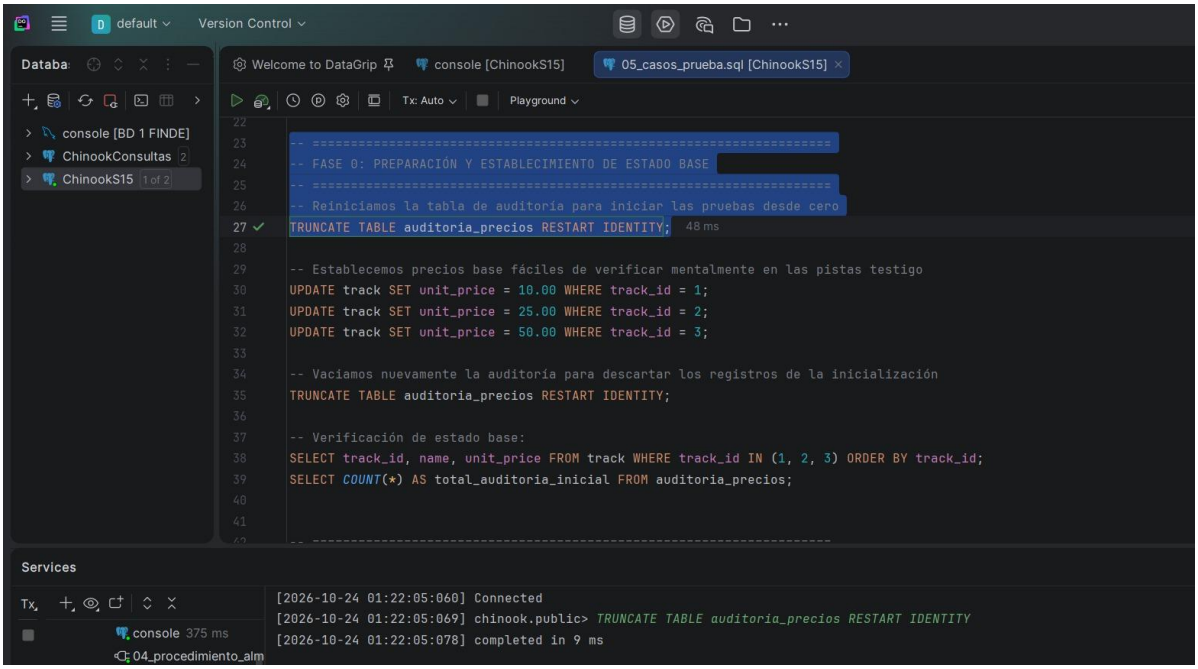


Figura 20: Limpieza inicial de la tabla auditoria_precios con TRUNCATE RESTART IDENTITY.

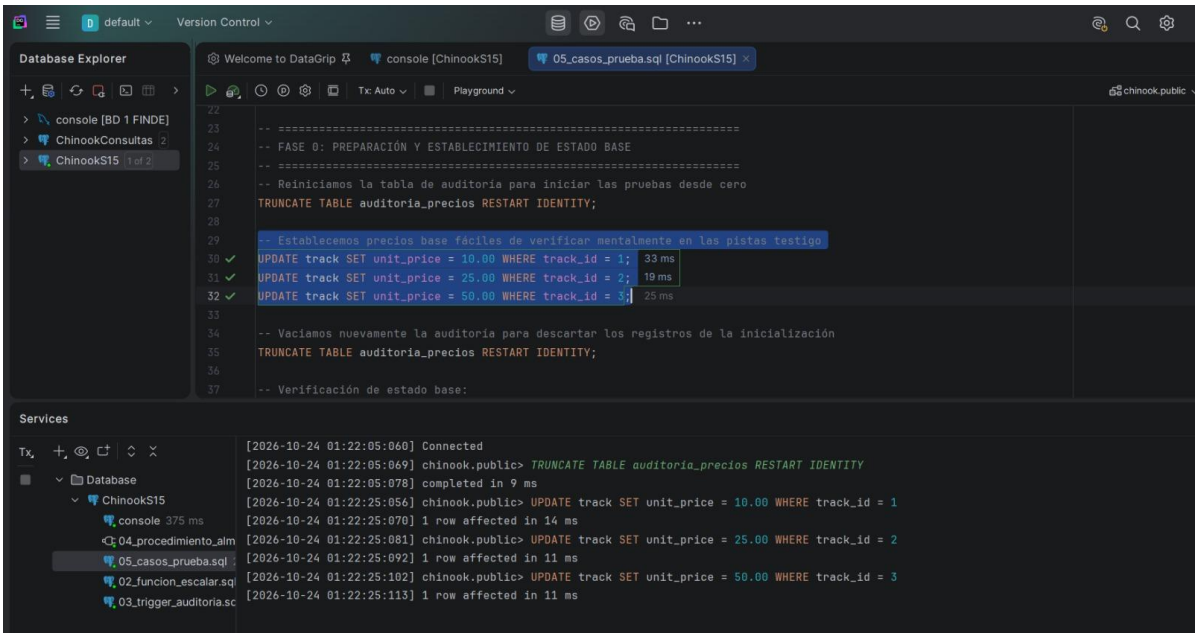


Figura 21: Establecimiento de precios testigo verificables mentalmente: Track 1 = 10.00, Track 2 = 25.00 y Track 3 = 50.00.

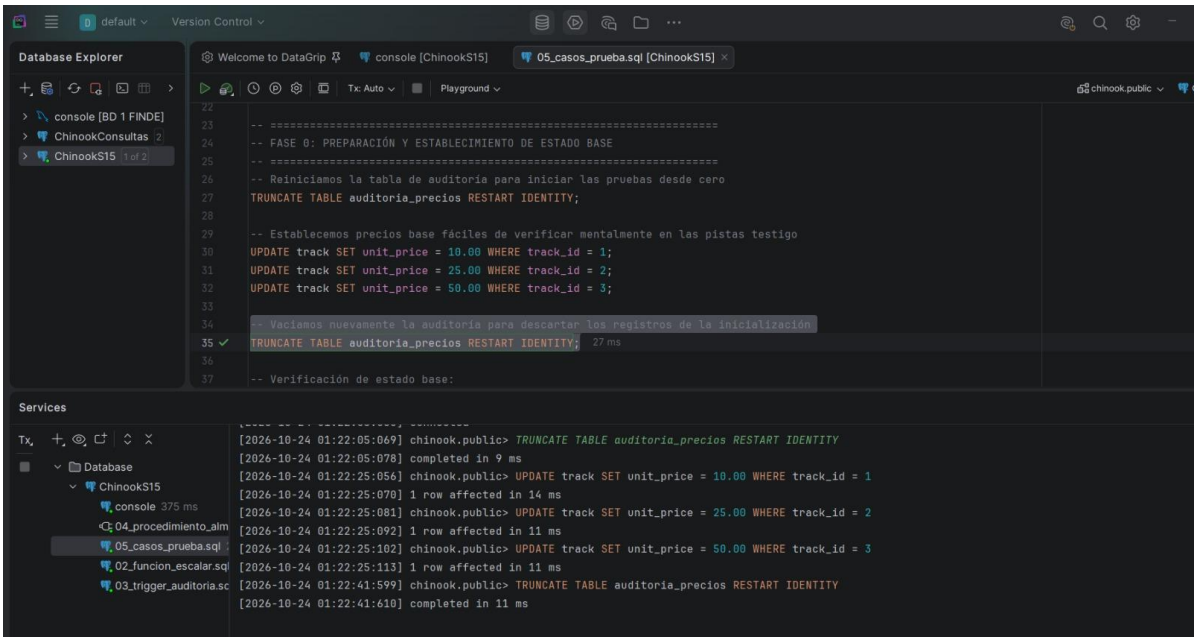


Figura 22: Vaciado de auditoría para descartar los registros generados por la inicialización de los precios base.

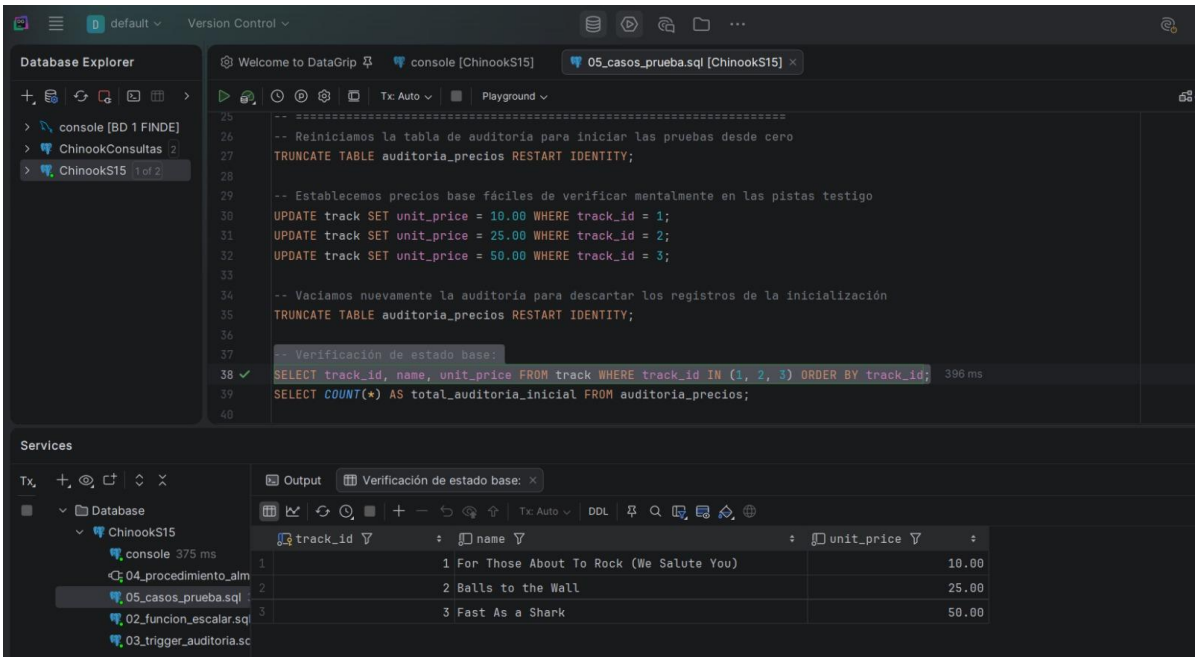


Figura 23: Comprobación del catálogo base listo: Pistas 1 (10.00), 2 (25.00) y 3 (50.00).

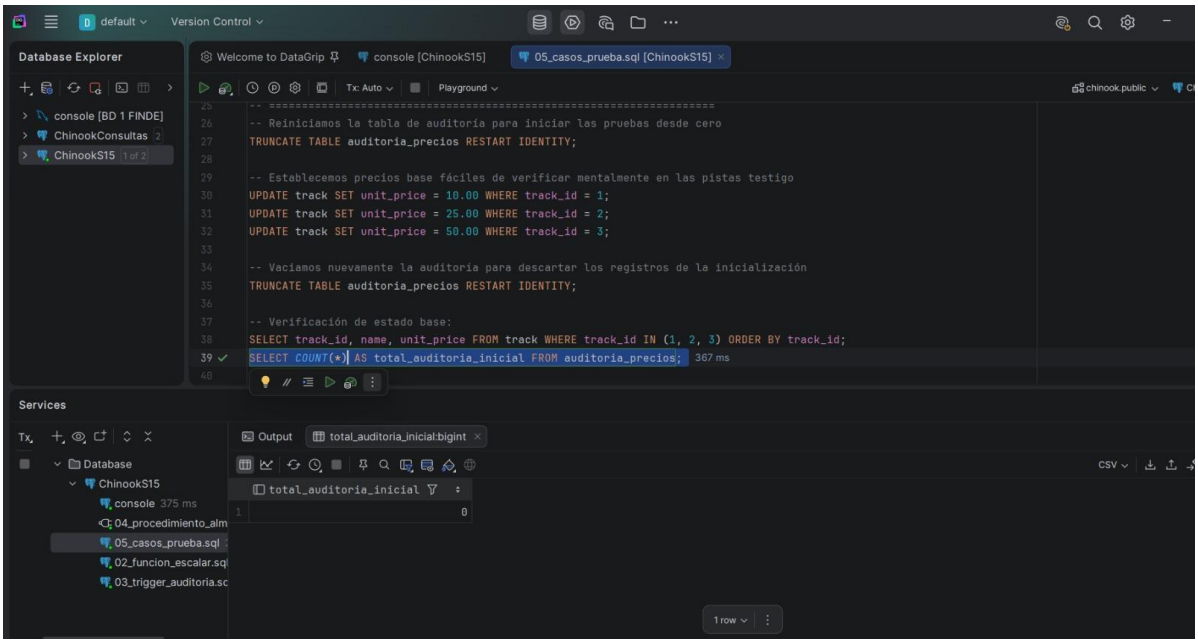


Figura 24: Verificación del conteo de auditoría en cero absoluto (total_auditoria_inicial = 0).

Propósito Técnico: Se fijaron deliberadamente precios base cerrados (10.00, 25.00 y 50.00) para las pistas testigo 1, 2 y 3, lo cual permite al evaluador comprobar la exactitud de los porcentajes y las reversiones de forma mental e inequívoca.

7.1. Caso 1: Función Escalar — Aumento (+10%) y Reducción (-50%)

Propósito: Validar que la función calcula correctamente tanto incrementos como reducciones sobre el límite admitido (-50%) y redondea con precisión a dos decimales.

Datos Utilizados: precio_base = 10.00, porcentaje = 10.00% | precio_base = 10.00, porcentaje = -50.00%.

Resultado Esperado: aumento: 11.00 | reduccion: 5.00.

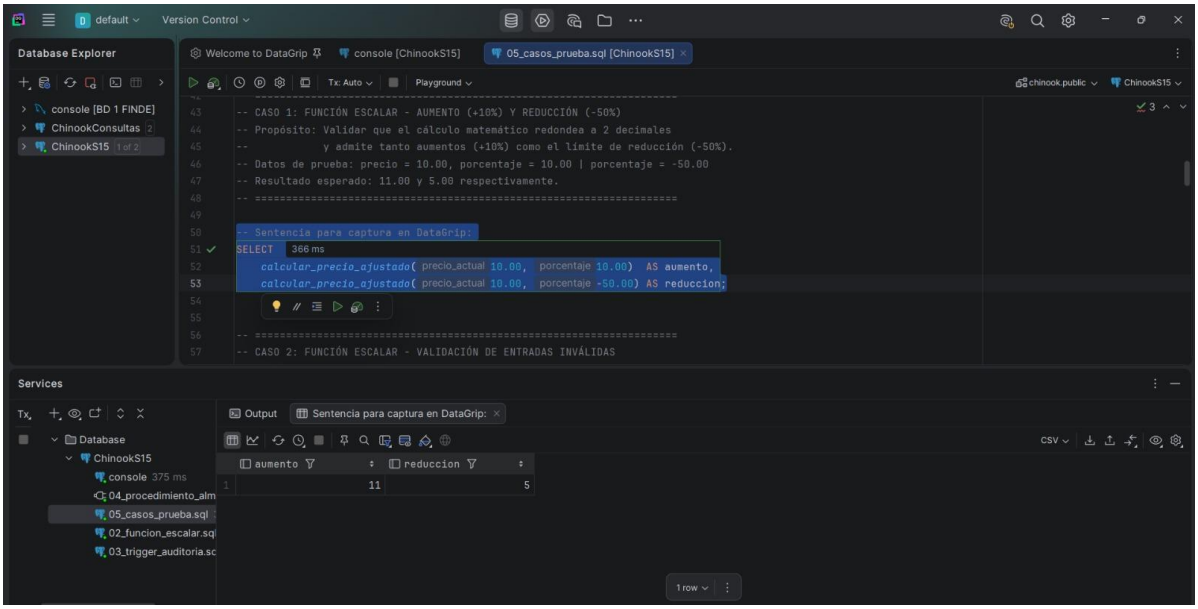


Figura 25: Ejecución de Caso 1 en DataGrip mostrando resultados de aumento = 11 y reduccion = 5.

Resultado Observado: La consulta SELECT calcular_precio_ajustado(10.00, 10.00) AS aumento, calcular_precio_ajustado(10.00, -50.00) AS reduccion; devolvió con exactitud 11.00 (aumento del 10%) y 5.00 (reducción del 50%), comprobando que el límite inferior negativo es procesado sin errores.

7.2. Caso 2: Función Escalar — Validación de Entradas Inválidas

Propósito: Verificar que la función rechaza de forma estricta parámetros NULL, precios menores o iguales a cero y porcentajes fuera del rango admisible [-50, 100].

Pruebas Realizadas: Se evaluaron 6 casos independientes ejecutados de manera individual en DataGrip:

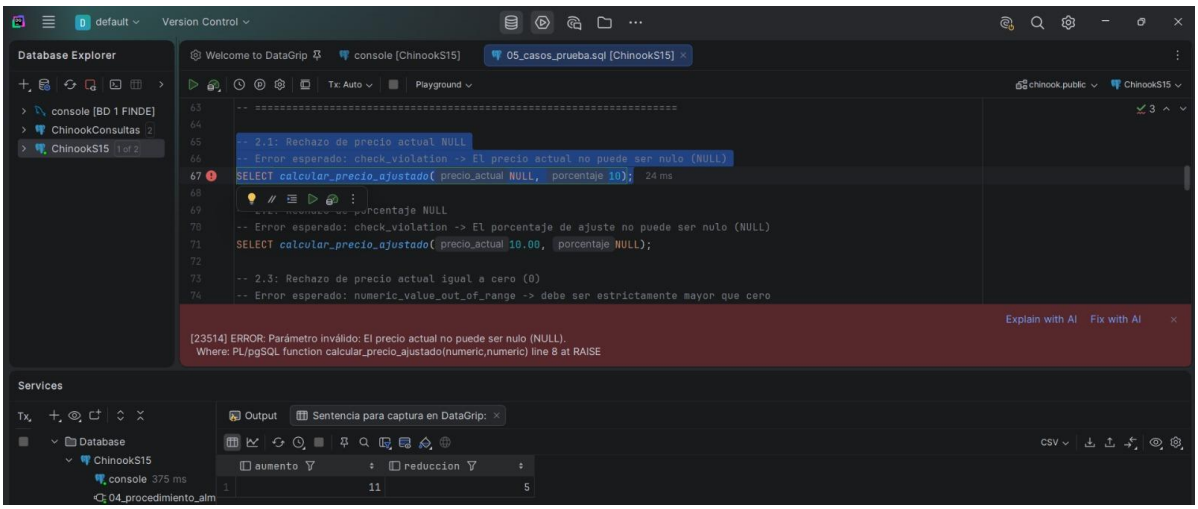


Figura 26: Prueba 2.1 — Rechazo de precio actual NULL con excepción [23514] check_violation.

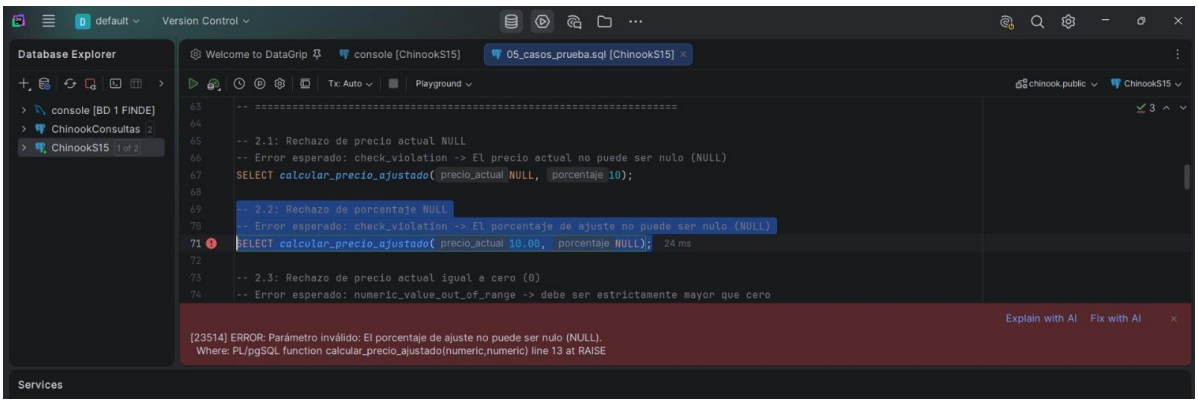


Figura 27: Prueba 2.2 — Rechazo de porcentaje de ajuste NULL con excepción [23514] check_violation.

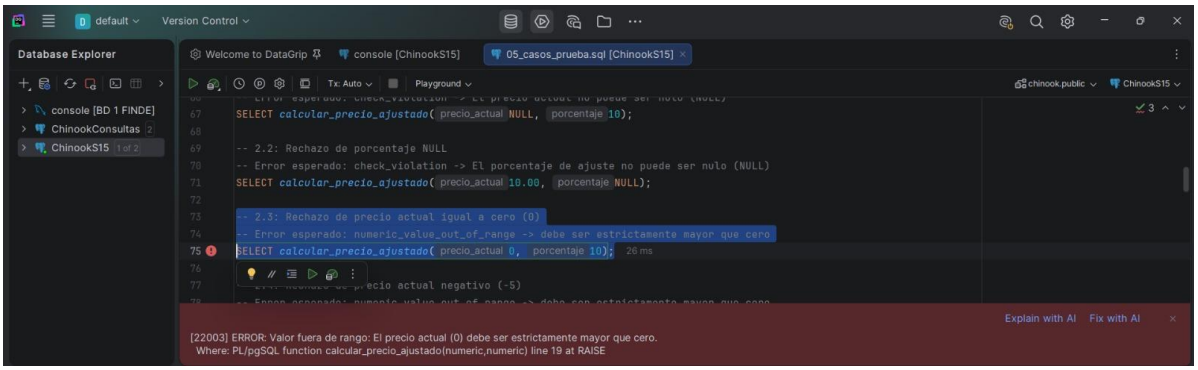


Figura 28: Prueba 2.3 — Rechazo de precio actual igual a cero (0) con excepción numeric_value_out_of_range.

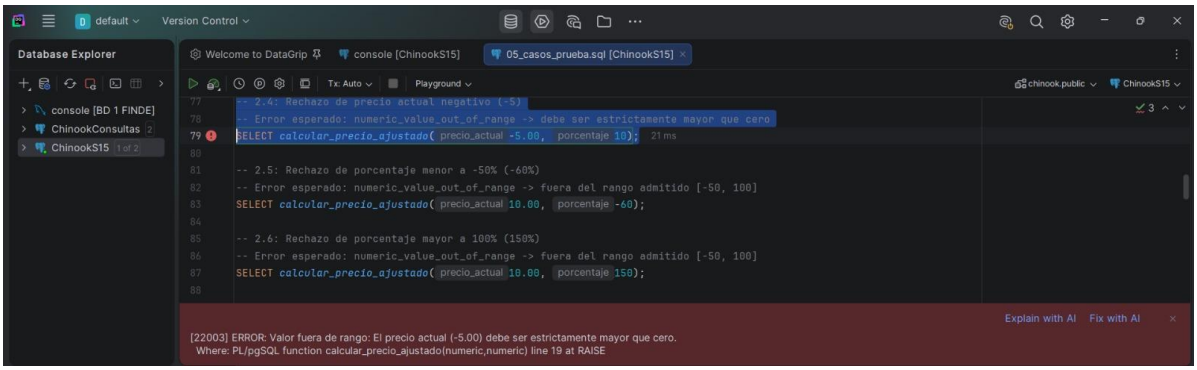


Figura 29: Prueba 2.4 — Rechazo de precio actual negativo (-5.00) con excepción numeric_value_out_of_range.

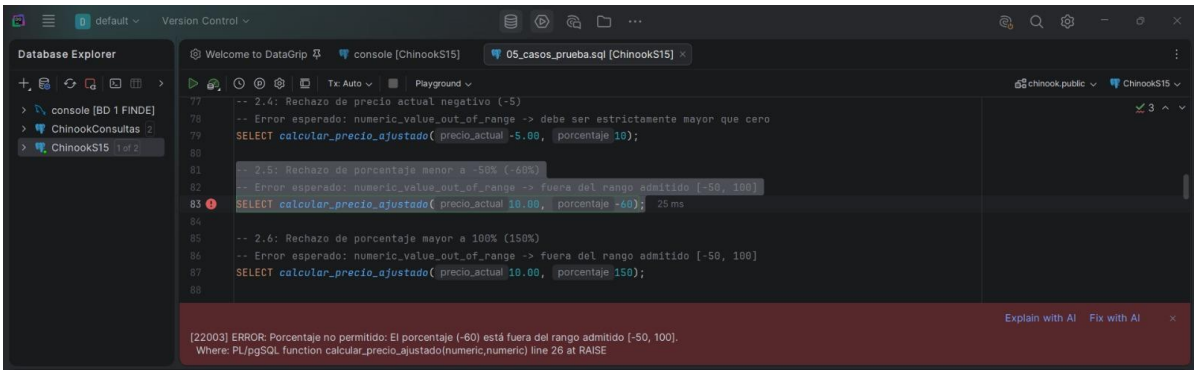


Figura 30: Prueba 2.5 — Rechazo de porcentaje inferior al mínimo permitido (-60.00%) fuera de [-50, 100].

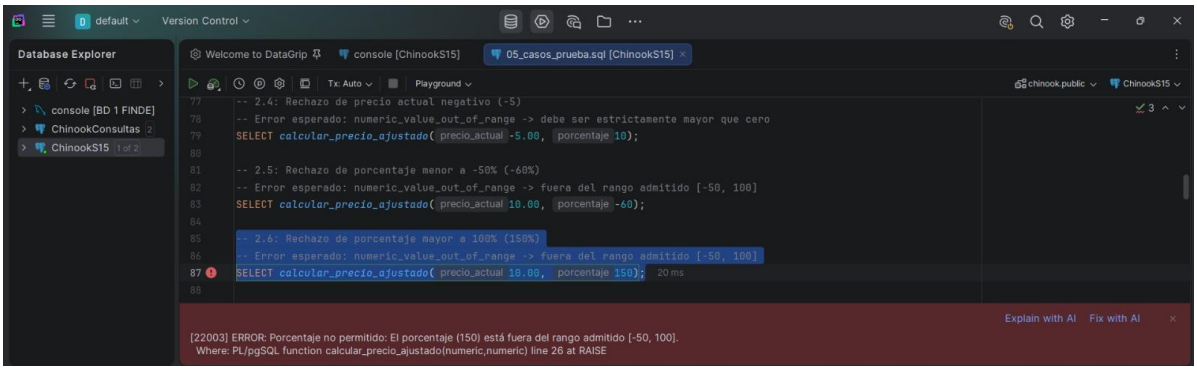


Figura 31: Prueba 2.6 — Rechazo de porcentaje superior al máximo permitido (150.00%) fuera de [-50, 100].

Resultado Observado: Cada una de las 6 pruebas generó el error esperado en la consola de DataGrip con el código SQLSTATE correspondiente y el mensaje descriptivo en lenguaje natural. Ninguna entrada anómala pudo evadir las precondiciones de la función.

7.3. Caso 3: Procedimiento Almacenado — Cambio Válido con CALL (+20%)

Propósito: Actualizar un producto existente (Track ID = 1) aumentando su precio en +20%. Verificar que el producto 1 cambia, el producto 2 permanece inalterado y el disparador genera automáticamente la fila de auditoría.

Datos Utilizados: p_id_producto = 1 (precio actual 10.00), p_porcentaje = 20.00% (incremento proyectado a 12.00).

Resultado Esperado: Track 1 se actualiza a 12.00; Track 2 permanece en 25.00; se crea una fila en auditoria_precios.

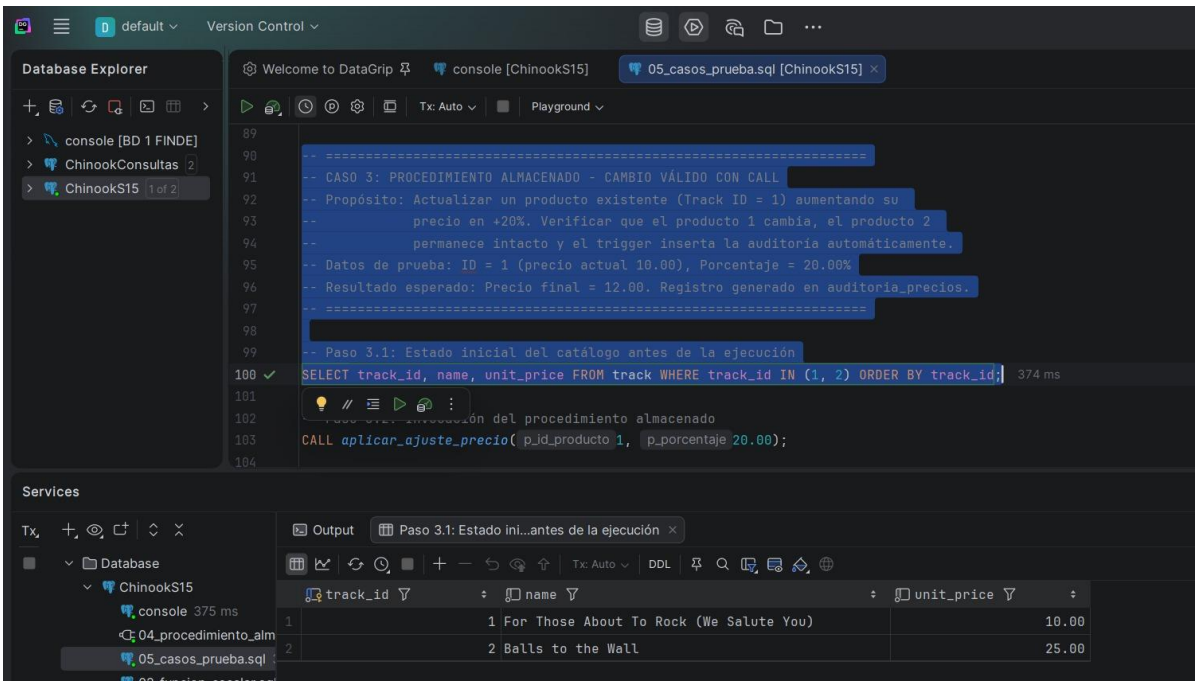


Figura 32: Paso 3.1 — Estado inicial del catálogo antes de la ejecución (Track 1 = 10.00, Track 2 = 25.00).

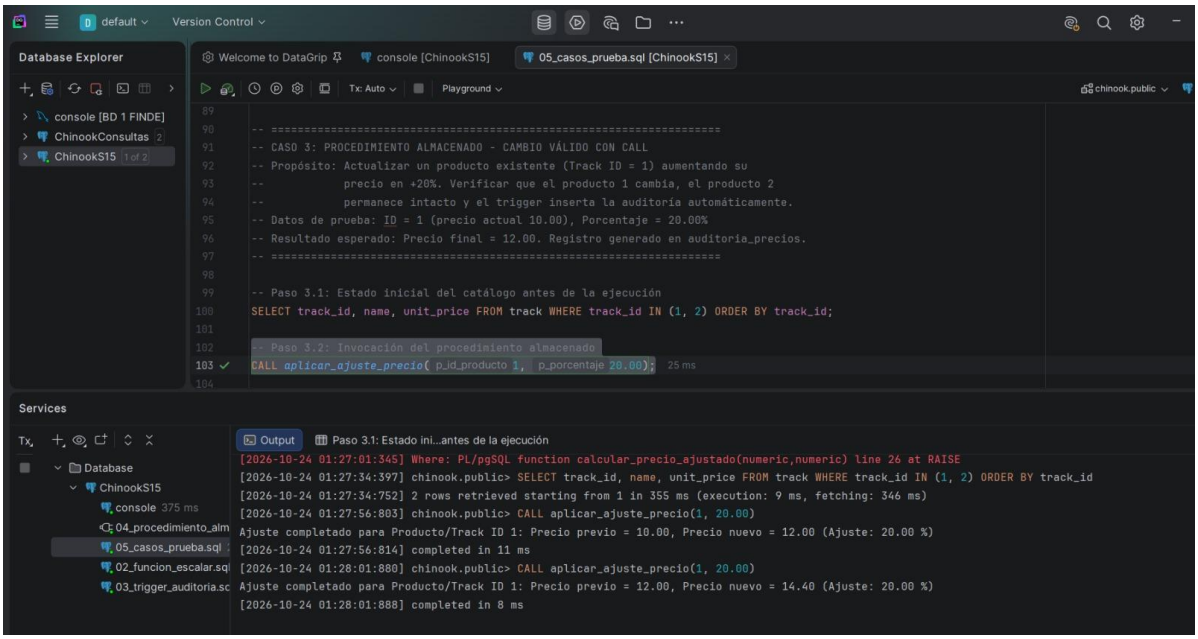


Figura 33: Paso 3.2 — Ejecución exitosa de CALL aplicar_ajuste_precio(1, 20.00) mostrando notificación RAISE NOTICE.

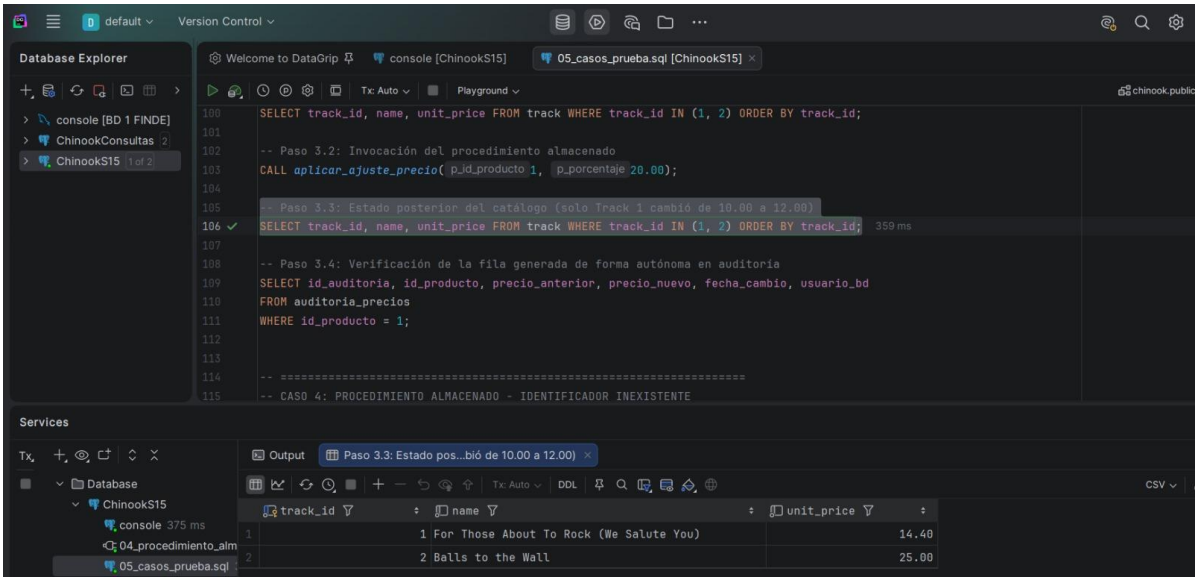


Figura 34: Paso 3.3 — Estado posterior del catálogo: Track 1 modificado y Track 2 inalterado en 25.00.

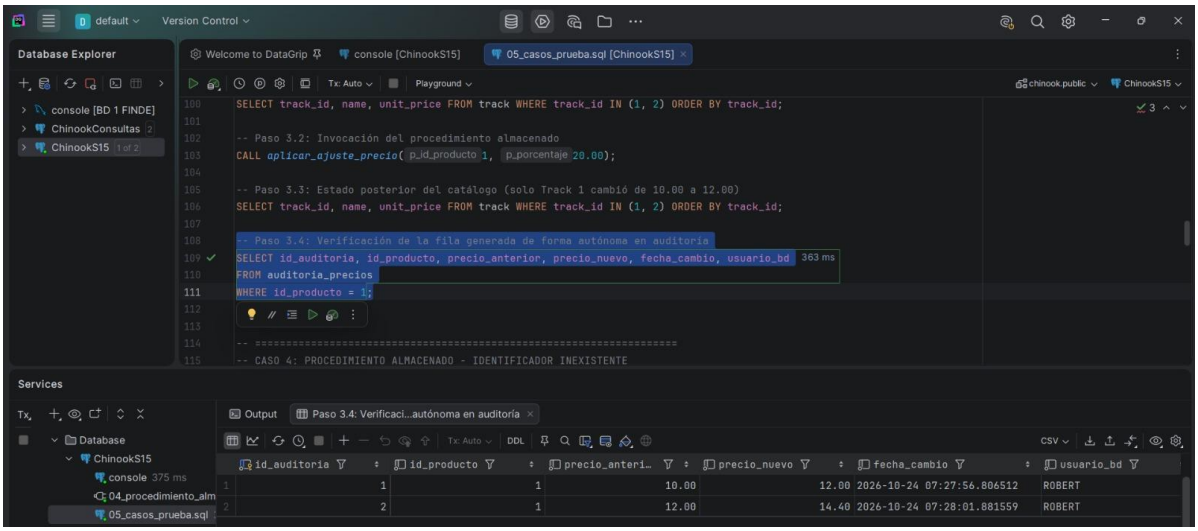


Figura 35: Paso 3.4 — Verificación en auditoria_precios del registro generado autónomamente por el trigger.

Resultado Observado: La invocación con CALL aplicar_ajuste_precio(1, 20.00) actualizó el precio de la pista 1 de 10.00 a 12.00, emitiendo el mensaje 'Ajuste completado para Producto/Track ID 1: Precio previo = 10.00, Precio nuevo = 12.00 (Ajuste: 20.00 %)'. La pista 2 se mantuvo inmutable en 25.00. La consulta a auditoria_precios demostró la inserción automática de la tupla con id_producto=1, precio_anterior=10.00, precio_nuevo=12.00 y usuario_bd='ROBERT'.

7.4. Caso 4: Procedimiento Almacenado — Identificador Inexistente

Propósito: Comprobar que al invocar el procedimiento con una clave primaria inexistente (Track ID = 999999), el sistema aborta de inmediato con una excepción controlada sin alterar datos del catálogo ni la bitácora.

Datos Utilizados: p_id_producto = 999999, p_porcentaje = 10.00%.

Resultado Esperado: Lanzamiento de excepción [22000] data_exception. Cero filas modificadas en track y cero filas añadidas a auditoria_precios.

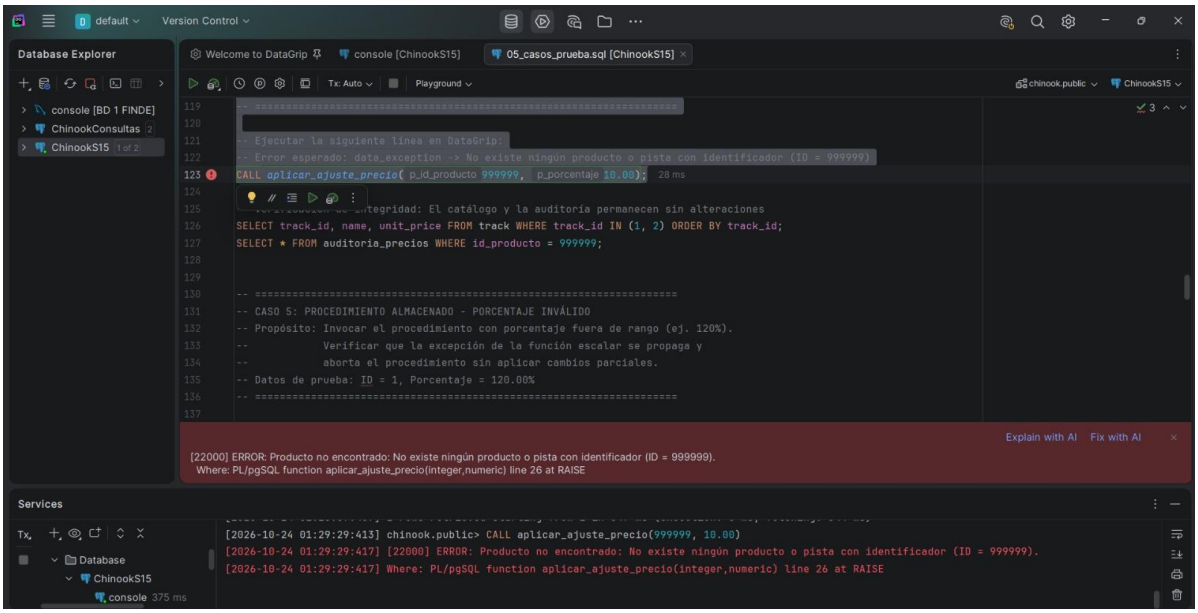


Figura 36: Ejecución de CALL aplicar_ajuste_precio(999999, 10.00) lanzando [22000] ERROR: Producto no encontrado.

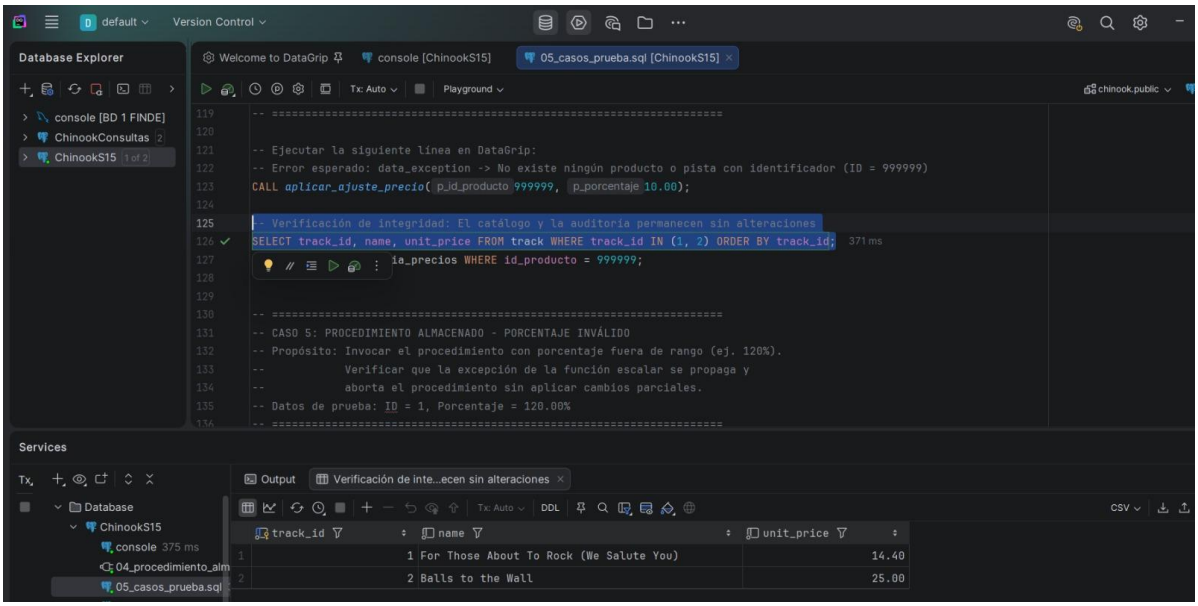


Figura 37: Comprobación de integridad en el catálogo: los productos 1 y 2 permanecen exactamente intactos.

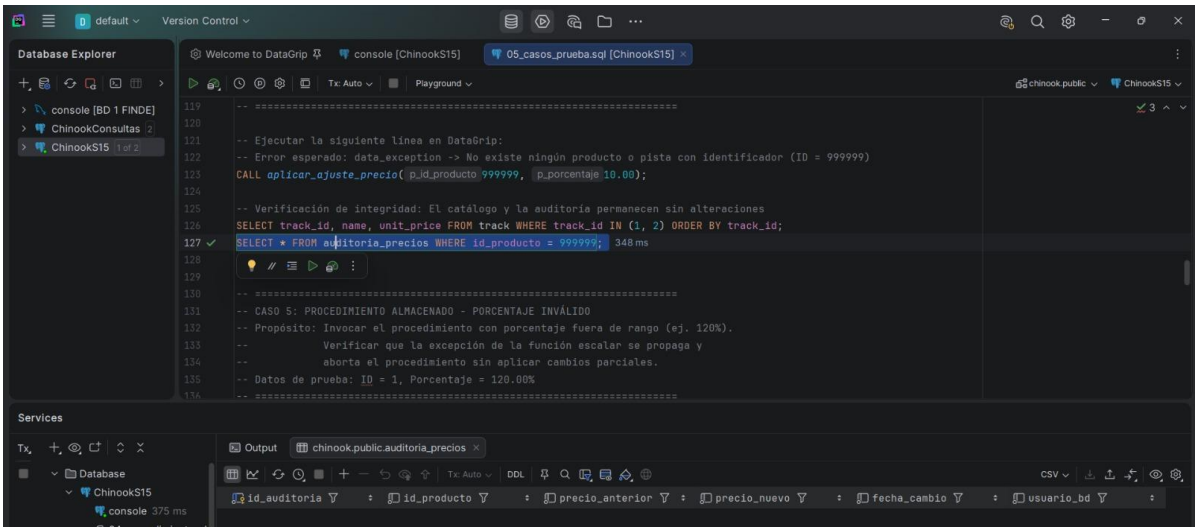


Figura 38: Consulta de auditoría para el ID 999999: resultado vacío (0 filas), demostrando ausencia de efectos secundarios.

Resultado Observado: El motor arrojó la excepción 'Producto no encontrado: No existe ningún producto o pista con identificador (ID = 999999)' con código 22000. Las Figuras 37 y 38 certifican que ni el catálogo de productos ni la tabla de auditoría registraron cambios.

7.5. Caso 5: Procedimiento Almacenado — Porcentaje Inválido (+120%)

Propósito: Demostrar que cuando se suministra un porcentaje fuera del rango admisible (ej. 120%), la función escalar propaga su excepción hacia el procedimiento orquestador, abortando la transacción sin aplicar cambios parciales en la base de datos.

Datos Utilizados: p_id_producto = 1, p_porcentaje = 120.00% (excede el límite superior del 100%).

Resultado Esperado: Excepción numeric_value_out_of_range propagada. Ausencia de cambios parciales en el precio de la pista 1.

Resultado Observado: Al ejecutarse CALL aplicar_ajuste_precio(1, 120.00), la invocación interna a calcular_precio_ajustado detectó que 120 > 100 y disparó la excepción 'Porcentaje no permitido: El porcentaje (120) está fuera del rango admitido [-50, 100]'. La ejecución se canceló de inmediato antes de llegar a la sentencia UPDATE, dejando el precio de la pista 1 intacto y sin generar registros en la tabla de auditoría.

7.6. Caso 6: Disparador (Trigger) — Actualización Directa vía UPDATE

Propósito: Demostrar la autonomía e ineludibilidad del disparador AFTER UPDATE: si un usuario u operador realiza un UPDATE directo en SQL sin invocar el procedimiento CALL, el trigger se activa automáticamente y registra la auditoría.

Datos Utilizados: Sentencia UPDATE directa sobre Track ID = 2 modificando su precio a 30.00 (precio anterior: 25.00).

Resultado Esperado: Inserción automática de una nueva fila en auditoria_precios con precio_anterior=25.00 y precio_nuevo=30.00.

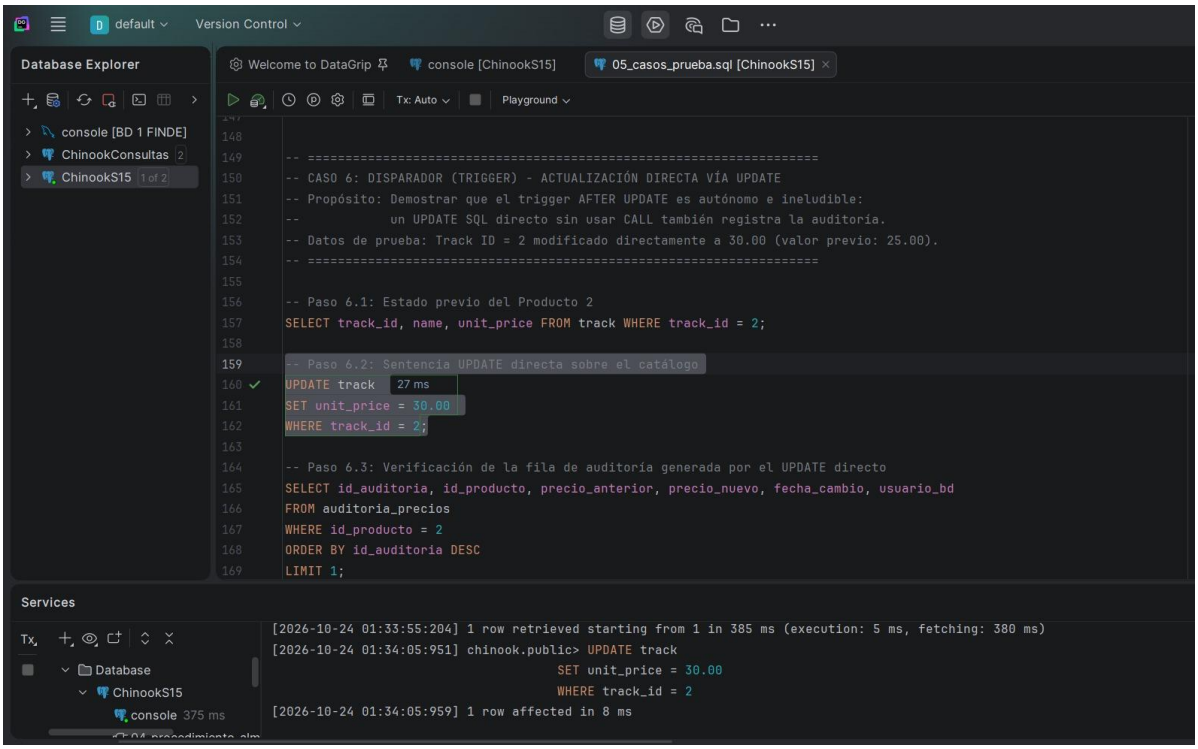


Figura 39: Paso 6.2 — Ejecución de sentencia UPDATE directa sobre Track 2 cambiando precio a 30.00 (1 row affected).

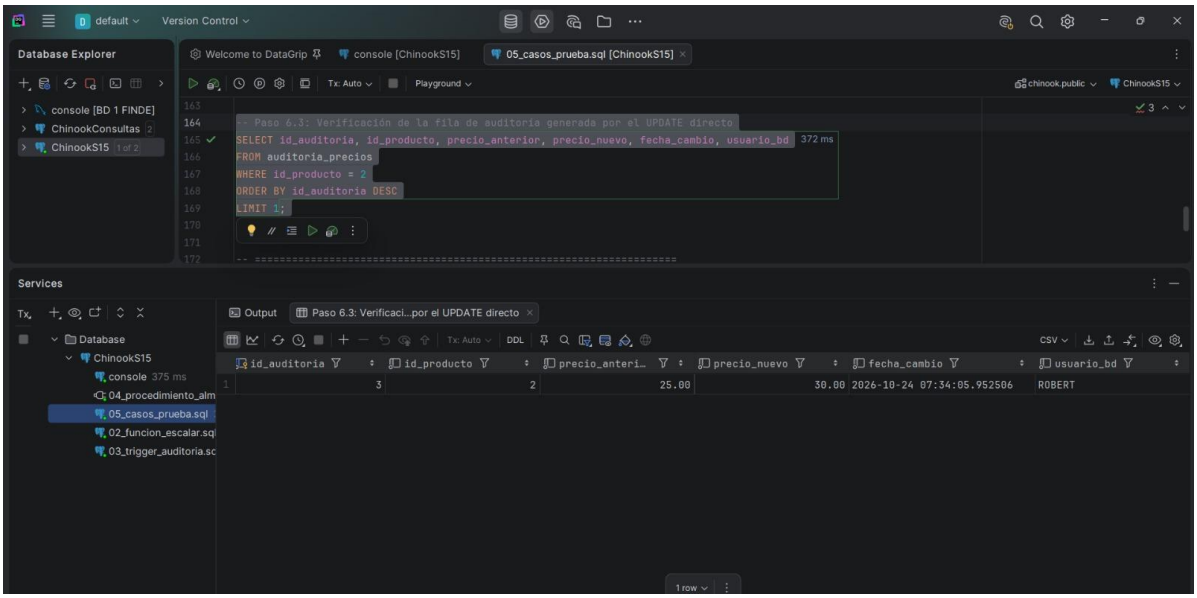


Figura 40: Paso 6.3 — Verificación de la fila de auditoría generada de forma autónoma por el disparador.

Resultado Observado: La Figura 40 comprueba que la tupla de auditoría con id_auditoria=3 fue creada automáticamente con id_producto=2, precio_anterior=25.00, precio_nuevo=30.00 y usuario_bd='ROBERT', demostrando que la auditoría no depende de que el invocador use el procedimiento almacenado.

7.7. Caso 7: Disparador (Trigger) — Precio sin Cambio Efectivo (Mismo Valor)

Propósito: Demostrar que gracias a la condición IS DISTINCT FROM, una sentencia UPDATE que asigne exactamente el mismo precio que ya posee el producto NO inserta ninguna fila redundante en la tabla de auditoría.

Datos Utilizados: UPDATE track SET unit_price = 30.00 WHERE track_id = 2; (su valor actual ya es 30.00).

Resultado Esperado: El conteo de filas en auditoria_precios antes y después de la operación debe ser exactamente idéntico.

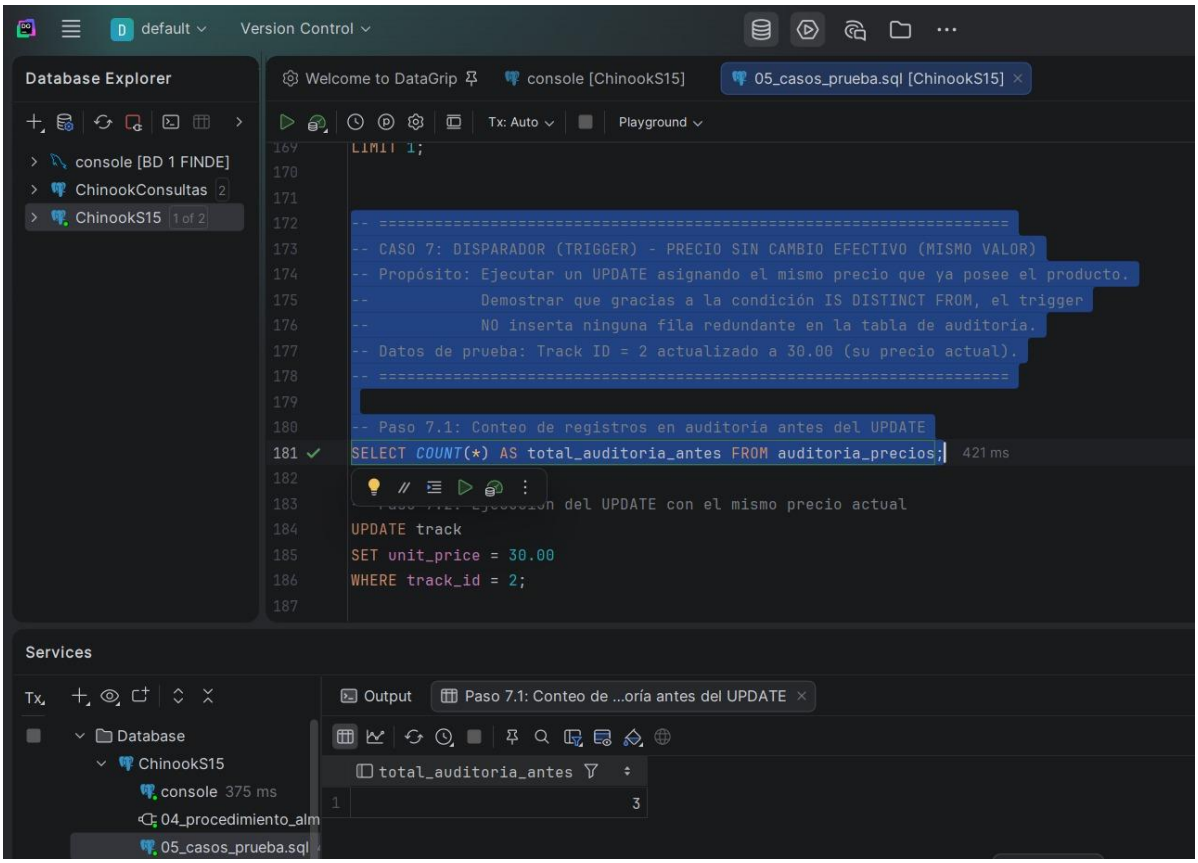


Figura 41: Paso 7.1 — Conteo previo en auditoria_precios (total_auditoria_antes = 3).

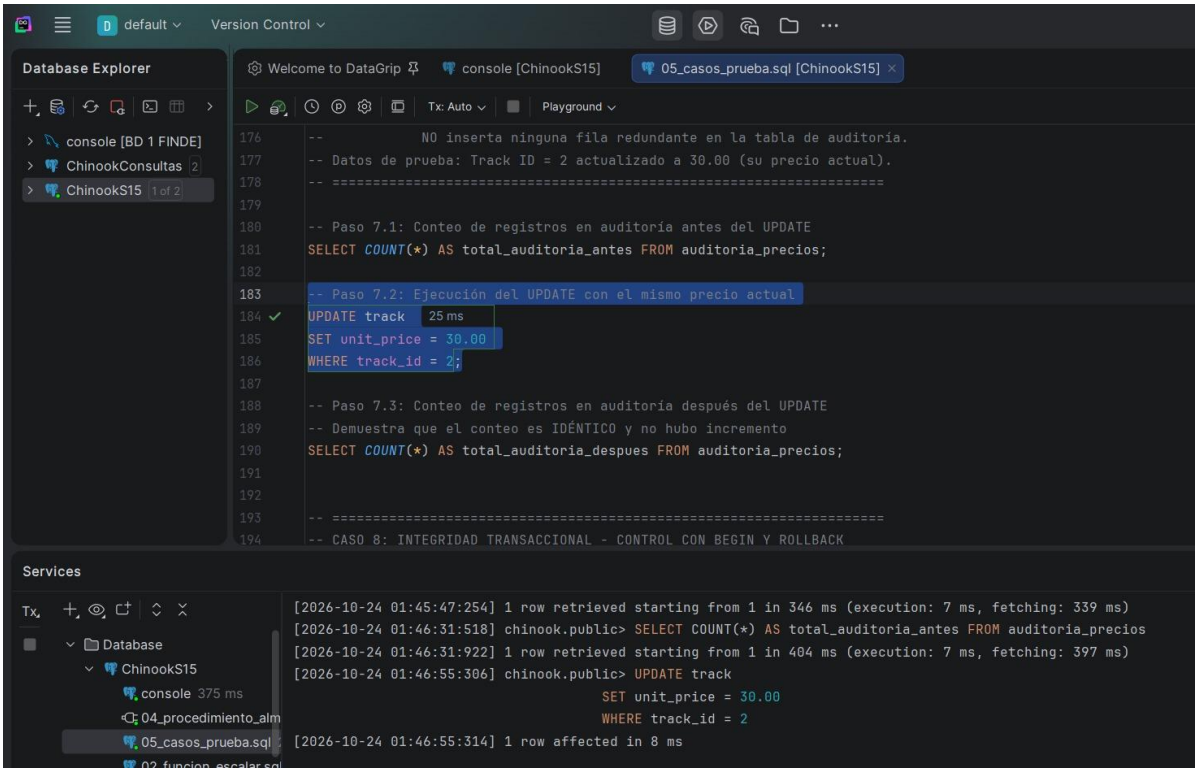


Figura 42: Paso 7.2 — Ejecución de UPDATE track asignando 30.00 (el mismo valor actual) con 1 row affected.

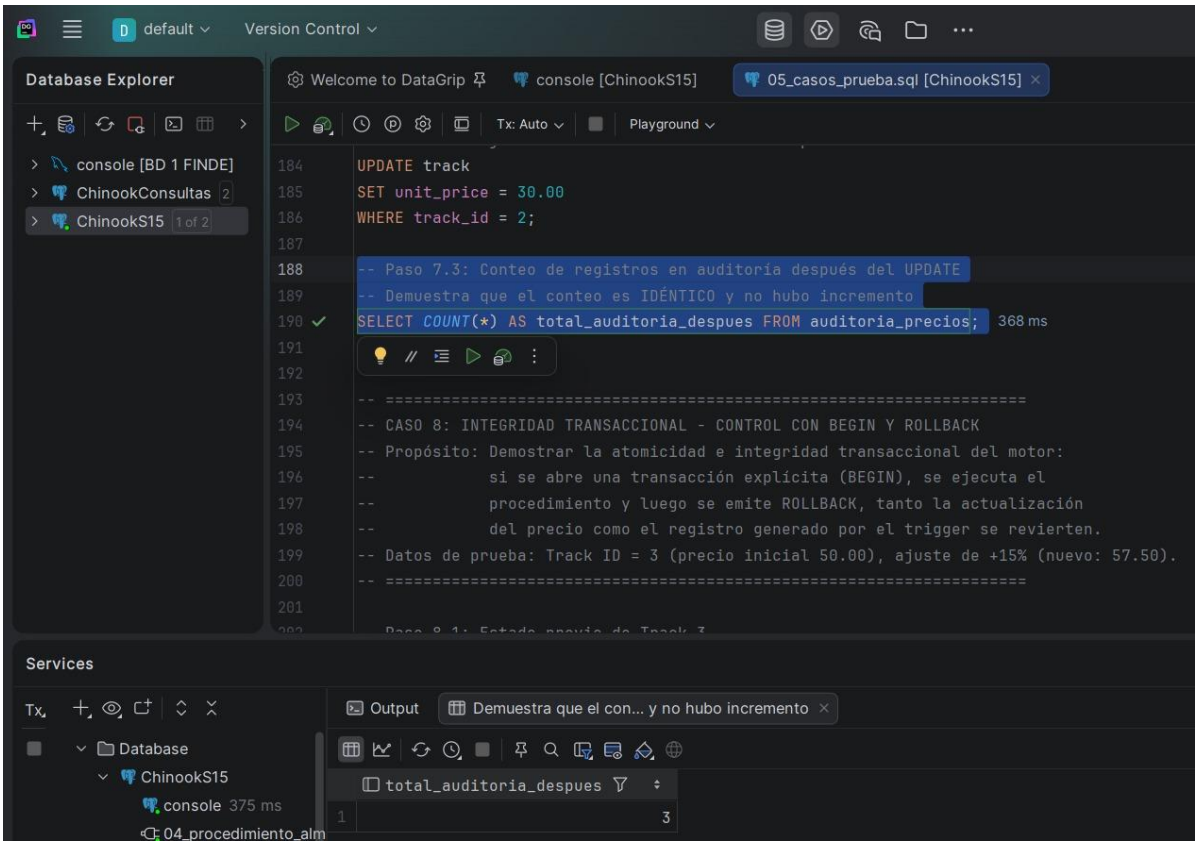


Figura 43: Paso 7.3 — Conteo posterior en auditoria_precios (total_auditoria_despues = 3, idéntico al anterior).

Resultado Observado: La evaluación de las Figuras 41 y 43 demuestra de manera concluyente que total_auditoria_antes fue 3 y total_auditoria_despues se mantuvo en 3. A pesar de que PostgreSQL procesó la fila en el UPDATE (1 row affected), el trigger evaluó OLD.unit_price IS DISTINCT FROM NEW.unit_price como FALSO y no generó ninguna fila basura. (Nota: se descartó image25.png por ser duplicado exacto de image24.png).

7.8. Caso 8: Integridad Transaccional — Control con BEGIN y ROLLBACK

Propósito: Demostrar la atomicidad del motor relacional mediante transacciones explícitas: al ejecutar un ajuste dentro de un bloque BEGIN y emitir posteriormente ROLLBACK, tanto la actualización de precio como la fila insertada por el trigger deben revertirse en su totalidad.

Datos Utilizados: Track ID = 3 (precio inicial 50.00), ajuste porcentual de +15.00% (precio en transacción: 57.50).

Resultado Esperado: Dentro de la transacción, el precio muestra 57.50 y la auditoría muestra la fila; tras el ROLLBACK, el precio regresa a 50.00 y la fila de auditoría desaparece.

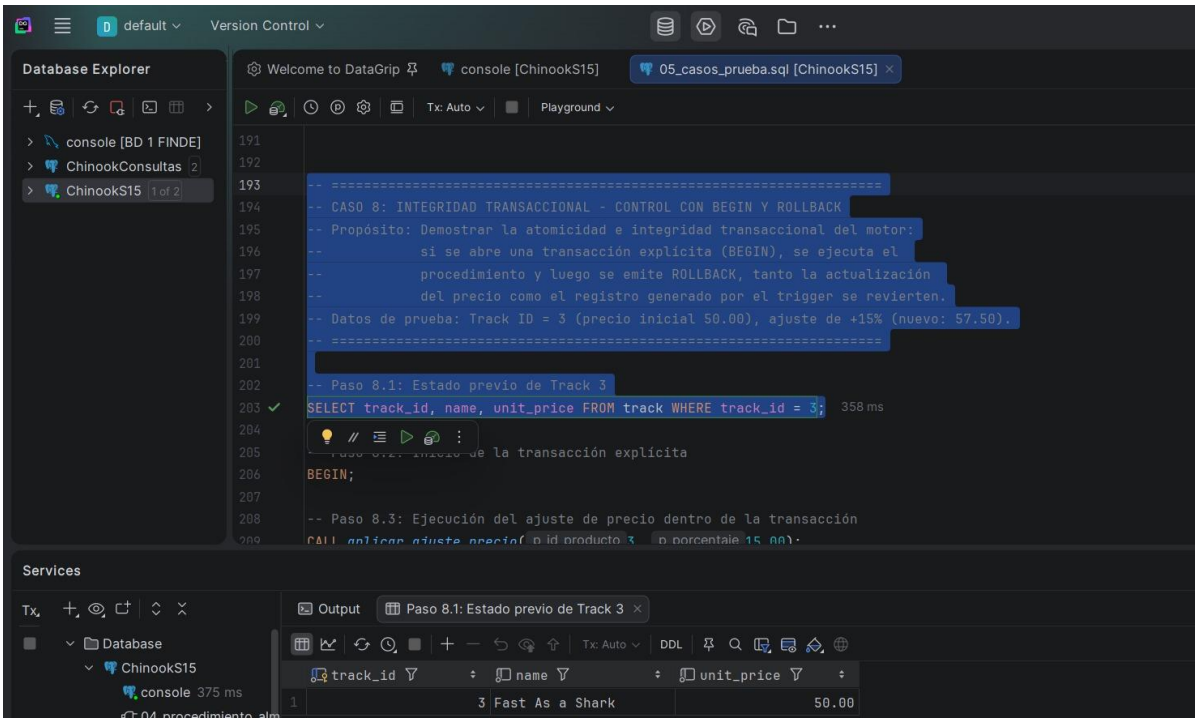


Figura 44: Paso 8.1 — Estado inicial de Track 3 en el catálogo (precio base = 50.00).

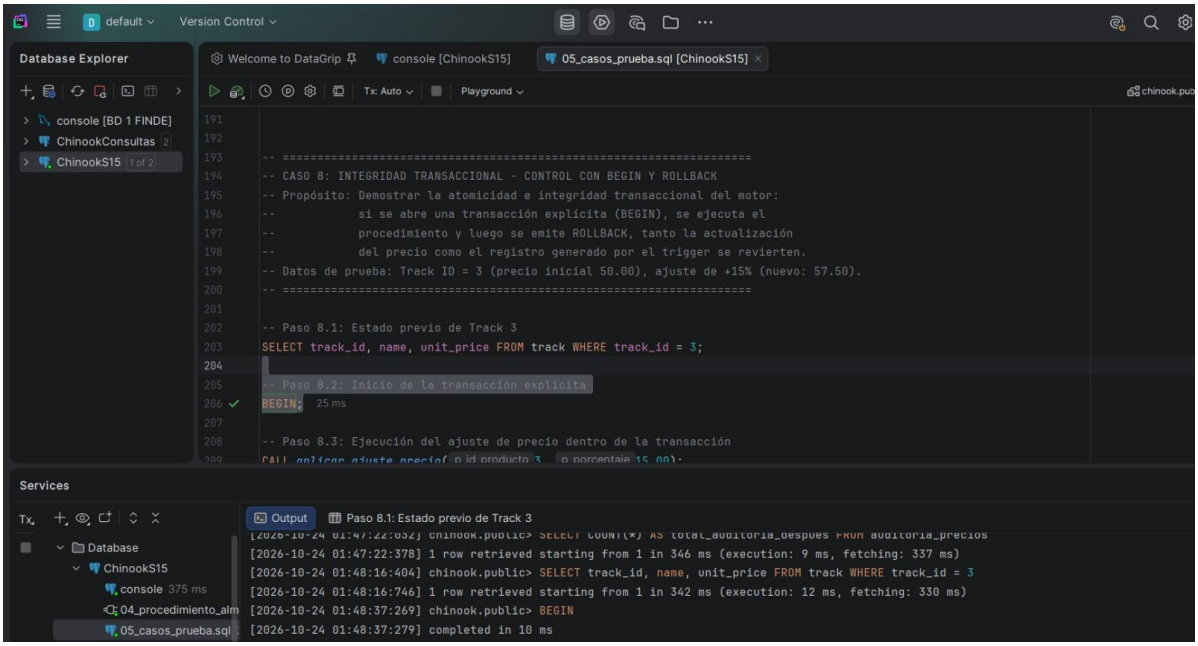


Figura 45: Paso 8.2 — Apertura de la transacción explícita mediante la sentencia BEGIN en DataGrip.

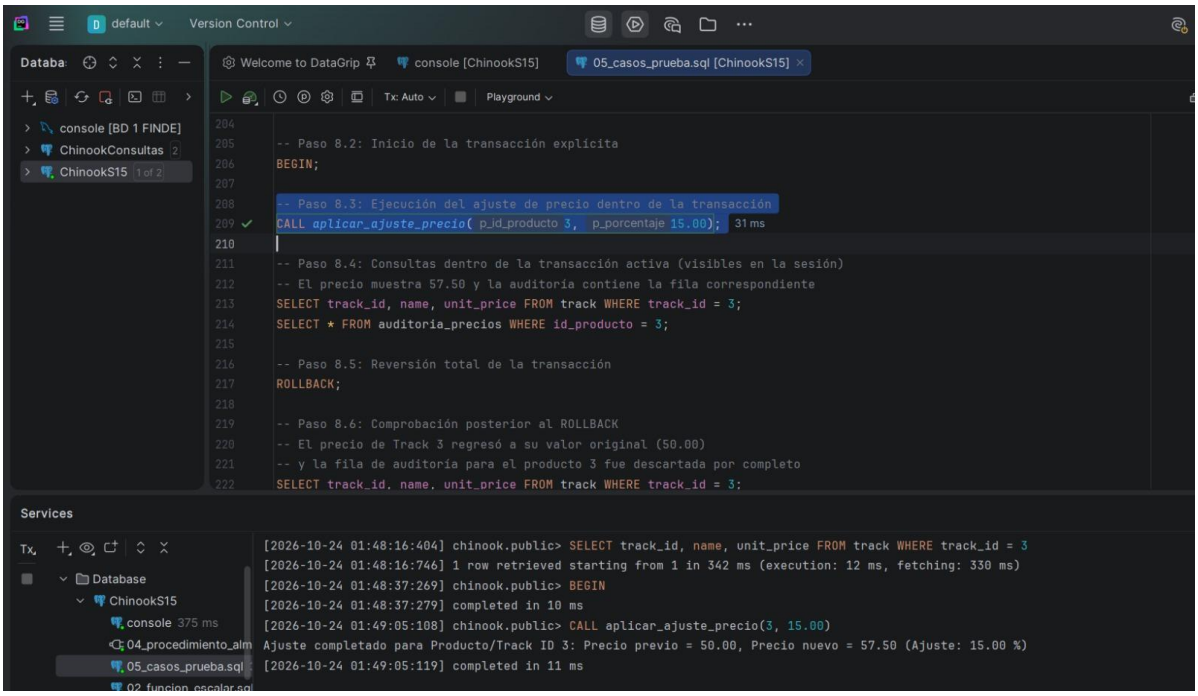


Figura 46: Paso 8.3 — Ejecución de CALL aplicar_ajuste_precio(3, 15.00) dentro de la transacción activa.

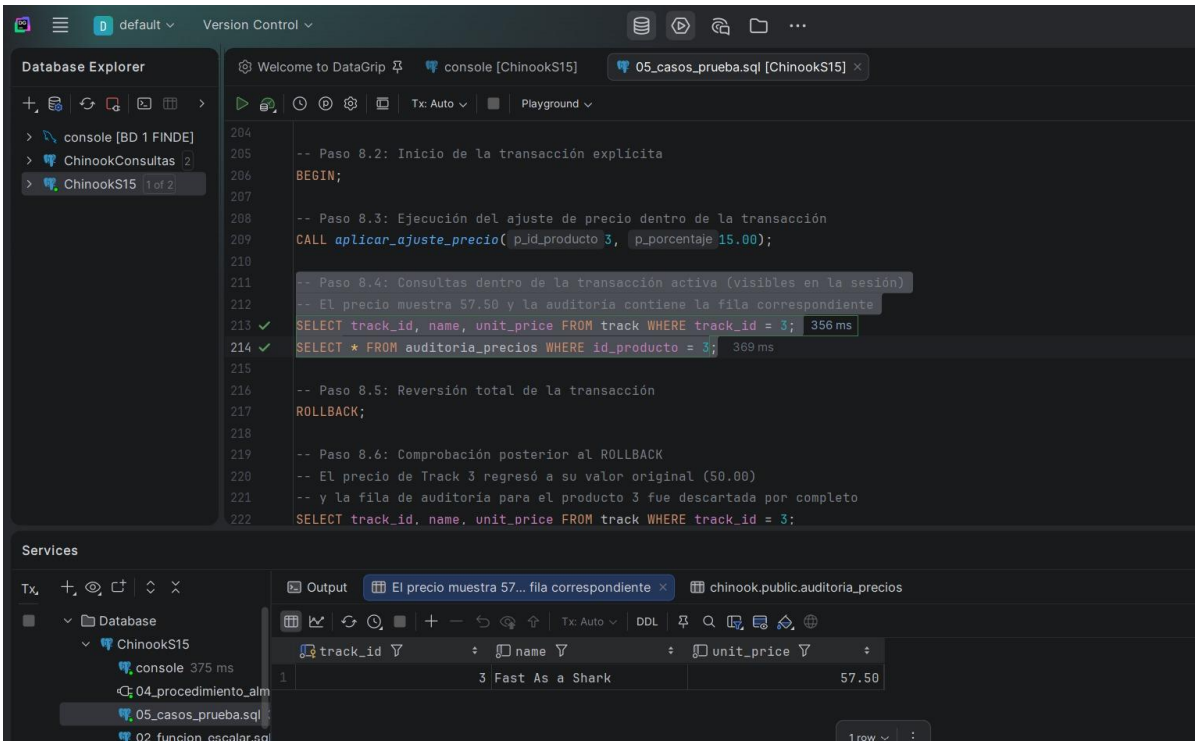


Figura 47: Paso 8.4 — Consulta del precio dentro de la sesión activa mostrando 57.50.

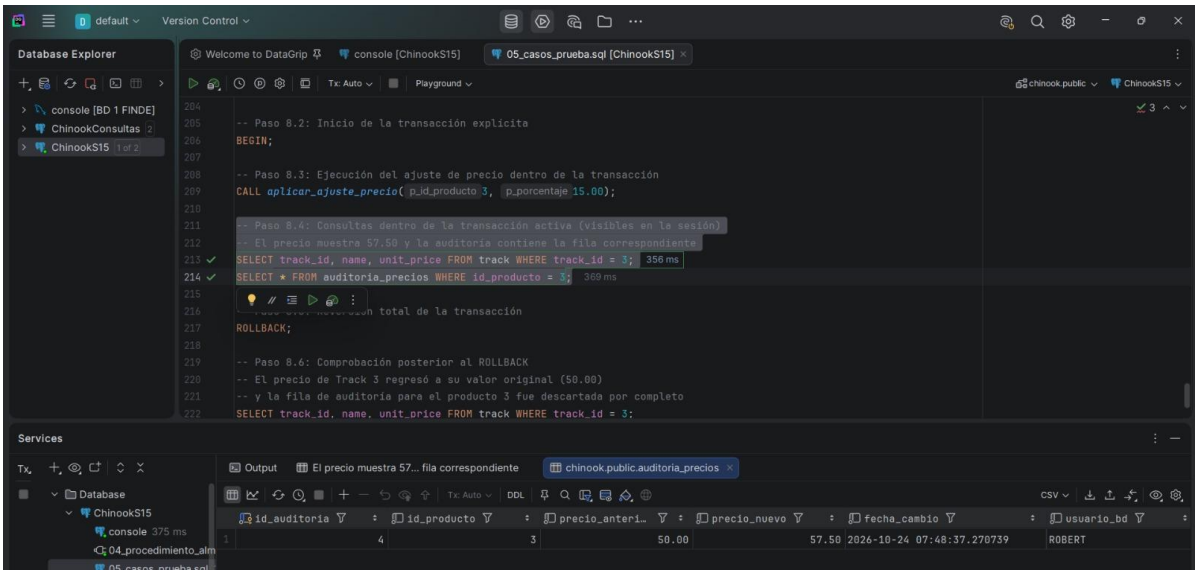


Figura 48: Paso 8.4 — Consulta de auditoría dentro de la transacción mostrando la fila generada para Track 3.

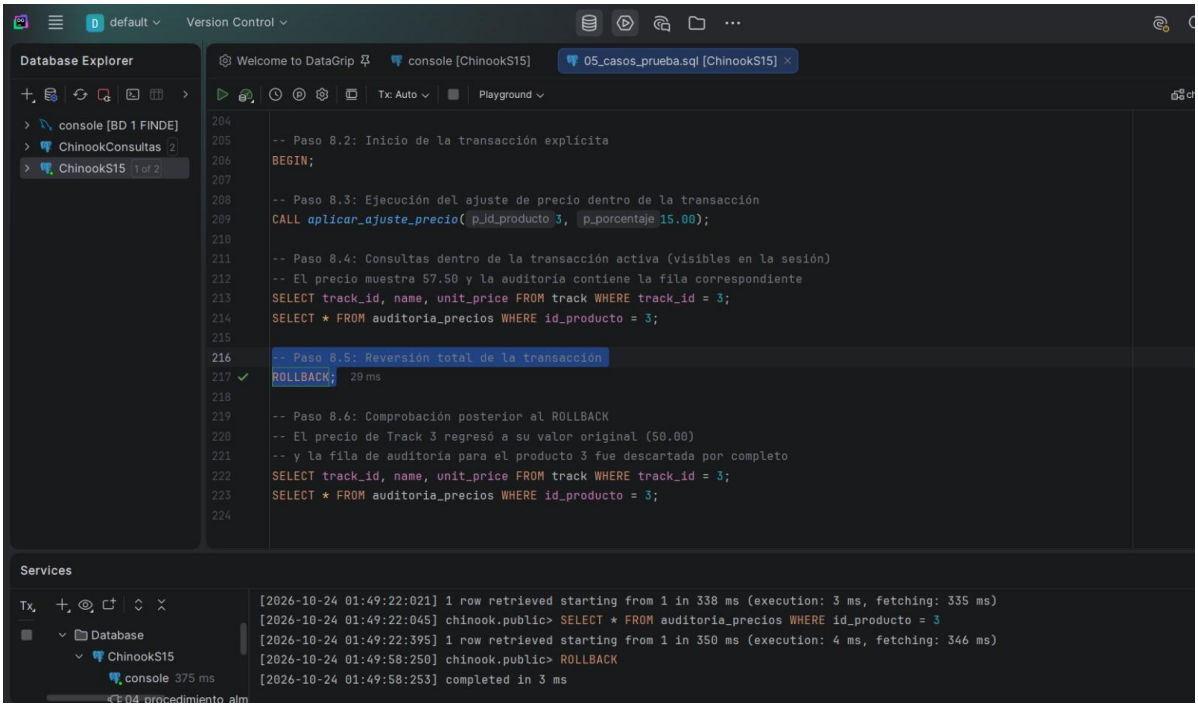


Figura 49: Paso 8.5 — Ejecución del comando ROLLBACK para abortar la transacción.

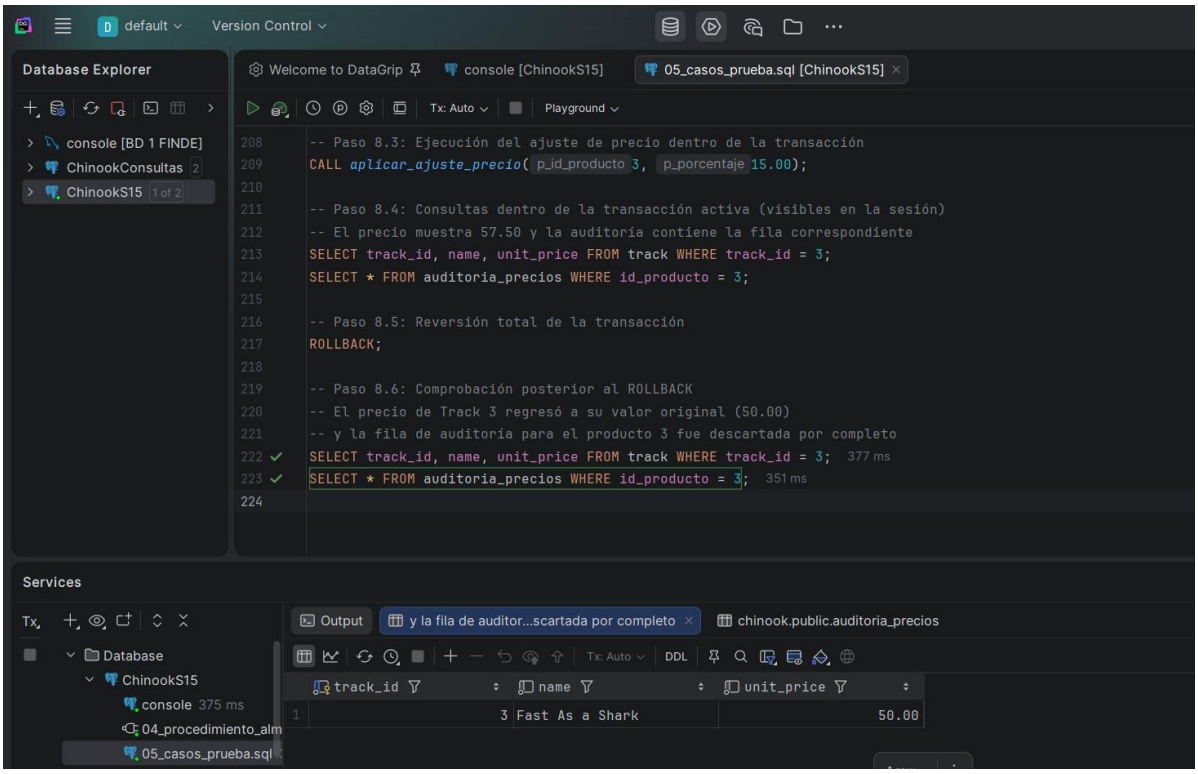


Figura 50: Paso 8.6 — Comprobación post-ROLLBACK en track: el precio de Track 3 regresó a su valor original de 50.00.

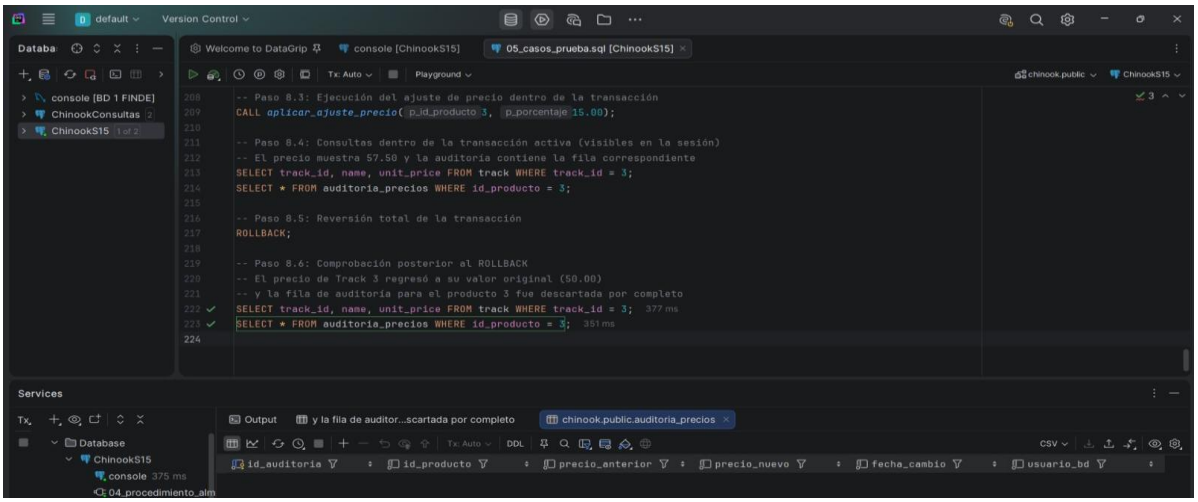


Figura 51: Paso 8.6 — Comprobación post-ROLLBACK en auditoria_precios: la fila fue descartada por completo (0 filas).

Resultado Observado: La secuencia de capturas de las Figuras 44 a la 51 evidencia el cumplimiento estricto del principio de Atomicidad (propiedad ACID). Durante la transacción, los cambios fueron visibles en la sesión (precio 57.50 y fila de auditoría id_auditoria=4). Al ejecutar ROLLBACK, PostgreSQL deshizo tanto la modificación en la tabla track como la inserción efectuada por el trigger en auditoria_precios, demostrando que ambos objetos operan dentro del mismo contexto transaccional coordinado.

Conclusiones Técnicas y Respuestas Teóricas

A partir del diseño, despliegue y experimentación rigurosa de los objetos de base de datos en PostgreSQL 18, se formulan las siguientes conclusiones fundamentales:

Diferenciación Estructural entre Objetos:
• Función Escalar: Objeto determinista, de cómputo puro y sin efectos colaterales. Acepta argumentos y devuelve un único valor atómico primitivo (NUMERIC), permitiendo su uso directo en expresiones SELECT, WHERE y asignaciones PL/pgSQL.
• Procedimiento Almacenado: Unidad de orquestación de negocio invocada con CALL. Su objetivo es coordinar validaciones complejas, interactuar con múltiples tablas y aplicar cambios en el estado de la base de datos (sentencias DML) bajo un control transaccional flexible.
• Disparador (Trigger): Mecanismo pasivo de seguridad reactiva y trazabilidad. Se ejecuta de manera automática e ineludible en el motor ante eventos de tabla (AFTER UPDATE), garantizando que las reglas de auditoría se cumplan sin importar la procedencia de la sentencia SQL.

¿Por qué la Auditoría no debe depender de que el usuario la ejecute?:
Si la inserción en la tabla de auditoría se delegara a una llamada manual en el procedimiento o a la buena fe de los desarrolladores de la aplicación cliente, existiría una vulnerabilidad arquitectónica crítica: cualquier actualización realizada directamente por el administrador vía consola, scripts de mantenimiento, herramientas BI o errores en el código de la app omitiría el registro histórico. Al implementar un disparador AFTER UPDATE en el motor, la auditoría se vuelve universal, obligatoria e inmutable, imposibilitando la alteración clandestina de precios sin dejar rastro.

Eficiencia y Mitigación de Redundancia con IS DISTINCT FROM:
En bases de datos de alto tráfico, las actualizaciones ciegas que reasignan los mismos valores preexistentes pueden saturar el disco y degradar el rendimiento con escrituras innecesarias en bitácoras. La comprobación OLD.unit_price IS DISTINCT FROM NEW.unit_price demostró ser la técnica óptima para filtrar actualizaciones idénticas a costo de cómputo mínimo.